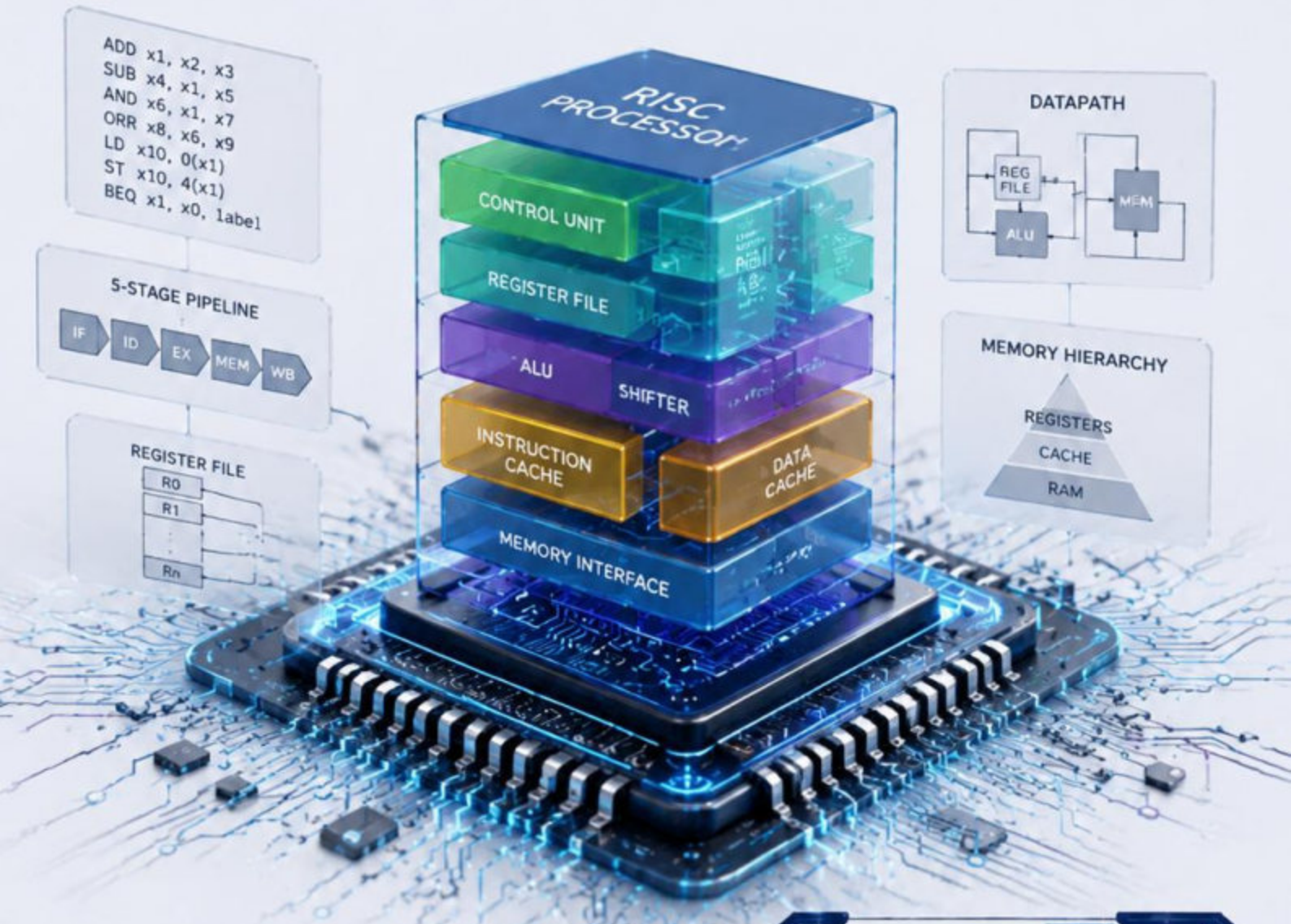
فريق نون التطوعي - السنة الرابعة

المعالجات الصغرية و نظمها

الملحق السابع :

معالجات RISC و المعمارية الدقيقة

قسم د.مازن محايري



كتابة : سالي السبسبي
تدقيق : عمر زين
تفريغ : حنين أحمد-ليلاس إدلبي

قسم هندسة الالكترونيات والاتصالات
2025 - 2026

الفصل الخامس: RISC(Reduced instruction set computer)

مقدمة:

في المحاضرات السابقة أنهينا الحديث عن معالجات انتل *(CISC)* التي تتميز ببساطة عتادها *Hardware*وتعقيد تعليماتها البرمجية *software* حيث في التعليمة الواحدة يلزم تنفيذ أكثر من أمر (التعليمة الواحدة تحتاج أكثر من نبضة ساعة)وفي هذه المحاضرة سنبدأ بنوع جديد من المعالجات وهي معالجات *RISC* التي تتميز بسهولة تعليماتها البرمجية(عدد التعليمات يزيد ولكنها تصبح أبسط (كل تعليمة تنفذ أمر واحد)) وتقتصر هذه التعليمات فقط على الجلب من الذاكرة والتنفيذ *(Load.Store)*و تتميز أيضاً بصعوبة تصنيع عتادها ولأجل هذا حدثت أنتل المعالجات لتصبح داخلياً تحول التعليمات البرمجية المعقدة إلى تعليمات برمجية بسيطة

نتيجة:

مقارنة بين معالجات *CISC,RISC*

CISC	*RISC*
عتاد بسيط	عتاد صعب التصنيع
تعليمات برمجية معقدة	تعليمات برمجية بسيطة
التركيز يكون على العتاد(يحوي المعالج واحدات فك تشفير للتعليمات المعقدة)	التركيز يكون على التعليمات البرمجية
تصاميم المعالجات غير مفتوحة المصدر	تصاميم المعالجات مفتوحة المصدر

ملاحظة: لايوجد معالج اسمه معالج *RISC* وأنما هي معمارية للمعالجات مفتوحة المصدر

سندرس بشكل تفصيلي الجيل الخامس من معالجات الريسك *RISC_V*

سؤال: ما الفرق بين *Architecture*و *Microarchitecture*؟

- *Architectur*: كيف يرى المبرمج الحاسوب (لا يدخل في تفاصيل العتاد)
- *Microarchitectute*: تنفيذ المعمارية كهاردوير

تعريفات:

- ✓ لغة الآلة: هي اللغة التي يفهما الحاسب دون وسيط(أصفار وواحدات)
- ✓ لغة التجميع: هي لغة وسيطة مقروءة للبشر يتكب فيها المبرمج ليتم ترجمتها من قبل المجمع *assembler* إلى أصفار وواحدات ليفهمها الحاسب

ملاحظة: تحول لغات البرمجة عالية المستوى مثل لغة C إما إلى لغة الآلة مباشرة أو يتم تحويلها إلى لغة التجميع ومن ثم إلى لغة الآلة.

معمارية RISC_V((الريسك فايف))

هي أول معمارية مفتوحة المصدر حيث يمكن لأي شخص، أو جامعة، أو شركة ناشئة تحميل التصميم وتعديله وصناعة معالج خاص بها دون دفع أي رسوم ترخيص.يحوي 32 مسجل طولها 32 بت

تم تطويرها من قبل كرستي أسانوفيتش(Krste Asanovic) وديفيد باترسون (David Patterson)وزملائهما في جامعة كاليفورنيا، بيركلي (UC Berkeley) عام 2010.

مبادى التي نعتمد عليها لتصميم المعمارية:

المبدأ الأول: البساطة تدعم الانتظام

كلما كان التصميم اكثر انتظاماً كان أبسط فمثلاً في لغة التجميع *RISC-VI* ، تجد أن جميع تعليمات الحساب والمنطق لها نفس عدد المعاملات بالضبط (ثلاثة معاملات: سجل الهدف، والسجل الأول، والسجل الثاني)، مثل *add x1, x2, x3.*كما أن حجم كل تعليمة ثابت دائماً وهو 32 بت.

المبدأ الثاني: جعل الحالة الشائعة سريعة

التركيز على تسريع العمليات التي تتكرر ملايين المرات في الثانية.

المبدأ الثالث: الأصغر هو الأسرع

كلما صغرت الدائرة الكهربائية أو قلّت سعة التخزين، كلما قلت المسافة التي يجب على الإشارات الكهربائية قطعها، وبالتالي قلّ زمن التأخير المستهلك في شحن وتفريغ السعات الكهربائية للأسلاك.(عدد المسجلات قليل 32 مسجل فقط)

ملاحظة: عدد مسجلات *RISC* أكبر بكثير من عدد مسجلات *CISC* ولكن بمعمارية *RISC_V* استعملنا 32 مسجل فقط .

المبدأ الرابع: التصميم الجيد يتطلب تنازلات جيدة:

أي نستعمل مبدأ المقايضة حيث أي قرار نتخذه لتحسين جانب ما سيكون على حساب جانب آخر.

تعليمة الجمع: *Add a,b,c*

Add هي التي تعطي أمر الجمع و*b,c* هي التعليمتان المراد جمعهما و ليتم تخزين النتيجة في *a*.

ملاحظة: كل التعليمات لها ثلاث معاملات

تعليمة الطرح: *Sub a,b,c*

Sub هي التي تعطي أمر الطرح و *b,c* هي التعليمتان المراد طرحهما و ليتم تخزين النتيجة في *a*.

في معالجات انتل كنا نتعامل مع أطوال تعليمات مختلفة ولكن في *RISC_V* كل التعليمات طولها 32 بت حصراً لأن هذا الحجم ثابت وعندما تكون التعليمات لها نفس الطول يكون توزيع البتات ثابت مما سيبسط عملية التشفير ويكون عدد المعاملات ثابت أيضاً حيث جميع تعليمات الريسك تأخذ الهيكلية التالية:

المصدر الأول, المصدر الثاني, الوجهة العملية

تتضمن معمارية *RISC_V* التعليمات شائعة الاستخدام فقط مما يصغر حجم العتاد اللازم لفك التشفير ومن أجل التعليمات المعقدة يتم تنفيذها بواسطة عدة تعليمات بسيطة

ملاحظة: تكون مواقع المعاملات الفيزيائية في الحاسوب بالمسجلات والذاكرة وممكن أن يكون المعامل عدد ثابت *immediate* مثال كما في تعليمة *Addi x1,x2,5* نريد جمع متغير مع عدد ثابت.



الموقع الأول من مواقع المعاملات: مسجلات RISC_V

- $Zero\ X_0$: مسجل له قيمة الصفر فقط له فوائد عند استخدام تعليمات النقل والتصفير
- $Return\ address\ (ra)\ X_1$: عندما ينفذ المعالج دالة، يقوم مسجل *ra* بحفظ عنوان الأمر التالي في البرنامج الأصلي ليعرف المعالج أين يرجع بدقة بعد انتهاء الدالة.

ملاحظة: وظيفة المبرمج هي حفظ العنوان ((تبسيط الهاردوير كان على حساب السوفت وير))

- $stack\ pointer\ (sp)\ X_2$: يشير إلى قمة "المكدس" في الذاكرة، وهو الجزء المسؤول عن تخزين المتغيرات المحلية أثناء استدعاء الدوال.
- $Global\ pointer\ (gp)\ X_3$ يستخدم للداتا الموجودة في كل البرامج مثل المتغيرات العامة
- $thread\ pointer(tp)\ X_4$: يستخدم للداتا الخاصة لكل برنامج مثل المتغيرات الخاصة

مثال: *tp* يلاحق اتجاه الماوس، *tp* يعرض صورة يعملان مع بعضهما ولا يؤثر عمل كل منهما على الآخر

- $saved\ registers/frame\ pointer(S_0/f_P)\ X8$: عندما نستدعي أكثر من دالة توضع معلومات كل دالة في إطار من الذاكرة إذا أرادت أي دالة استخدام أحد هذه المسجلات، فهي مجبرة على حفظ قيمته الأصيلة أولاً في المكدس *(Stack)* وإعادتها كما كانت قبل أن تنتهي *(Callee-saved).* هذا يضمن للبرنامج الرئيسي عدم ضياع بياناته الهامة. (لا يفضل استخدامها في البرامج الجزئية بل يفضل تمريرها عن طريق *arguments*)

((الزبدة: لما بدنا نستخدم نستخدم مسجل *saved* ببرنامج جزئي لازم احفظ معلومات البرنامج الرئيسي في المكدس))

- $Saved\ register\ (S_1)X_9$
- $Function\ arguments/return\ value\ a0-1$: مسؤول عن تمرير قيم المتحولات في دالة وإعادة قيمة النتيجة (وظيفته الرئيسية إعادة قيمة النتيجة)

ملاحظة: المبرمج هو من يمرر القيم.

- $Function\ arguments\ a2-7$: تمرير قيم المتحولات إلى دالة
- $saved\ registers\ X18-27$: وضع الداتا في إطار من الذاكرة

- $Temporaries\ X28-31$: وهي المسجلات المؤقتة التي يمكن أن نحتاجها في بعض العمليات للاحتفاظ بالداتا بكل مؤقت ((مثال في عملية الضرب نحتاج إلى عدة مراحل للحصول على الناتج)) الدالة المستدعاة تملك الحق في مسح وتعديل هذه المسجلات دون أي قيود، وعلى البرنامج الأصلي ألا يتوقع بقاء بياناته فيها بعد استدعاء أي دالة.*(Caller-saved)* ((لا يفضل أبدا استخدامها لحفظ معلومات دالة في البرنامج الرئيسي مشان ما تضيع))

تعليمة الجمع باستخدام المسجلات:

- ✓ الكود بلغات البرمجة عالية المستوى: *a = b+ c;*
- ✓ الكود بلغة التجميع:

```
#s0 = a, s1 = b, s2 = c add s0, s1, s ((اجمع b مع c وحط النتيجة بالمسجل(احفظها بالمسجل S)
```

ملاحظة: # في لغة التجميع تستخدم لادراج تعليق

تعليمة *addi*: تستخدم لجمع متغير مع عدد ثابت

- ✓ الكود بلغات البرمجة عالية المستوى: *a = b+6;*
- ✓ الكود بلغة التجميع:

```
#s0 = a, s1 = b, s2 = c addi s0, s1, s ((لاحظوا انه ما حطينا الرقم ب أي مسجل لأن العدد الثابت هون من أصل التعليمة ))
```

ولكن!!! البيانات كثيرة جداً بحيث لا يمكن احتواؤها في 32 مسجلاً فقط من أجل ذلك نلجأ إلى الذاكرة لتخزين المزيد من البيانات

ولكن!!! تخزين المعلومات في الذاكرة يكون على حساب السرعة(سيكون تخزين بطيء لهذا السبب نحاول تجنبها).

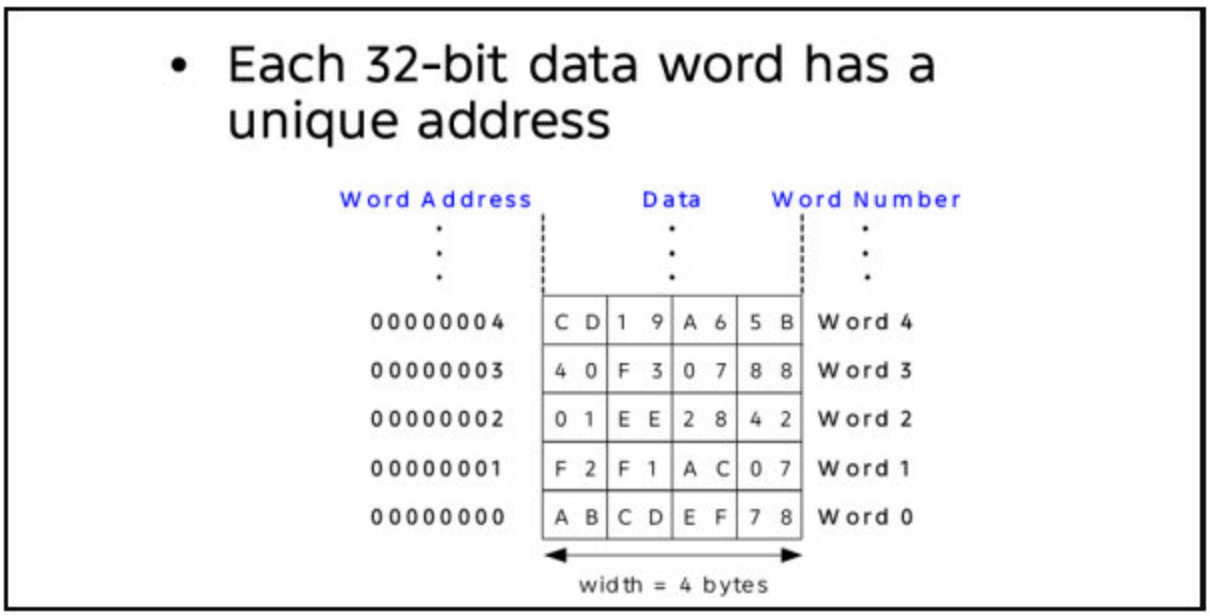


الموقع الثاني: الذاكرة

العنونة: لدينا نوعين من العنونة في الذاكرة:

- **العنونة بالكلمة *Word addressable*:**

عرض الكلمة 32 بت ويتم عنونة الكلمة الواحدة في موقع واحد (عنوان واحد)



العيوب: لو أراد المعالج قراءة أو تعديل حرف واحد فقط (والحرف يمثل بـ 1 بايت = 8 بت)، فإنه مضطر لجلب الـ 32 بت كاملة ثم فرزها برمجياً، مما يسبب بطئاً في التعامل مع البيانات الصغيرة.

القراءة بالذاكرة ذات العنونة بالكلمة:

تسمى عملية القراءة من الذاكرة بالتحميل *load word (lw)*

التعليمة:

```
Lw t1,5(s0)
Lw destination, offset(base)
```

معنى تعليمة load احضر من الذاكرة، وضع في المسجل t1 الذي عنوانه s0+5

ملاحظة: يمكن استخدام أي مسجل ليحوي العنوان الأساسي *base*.

الكتابة بالذاكرة ذات العنونة بالكلمة:

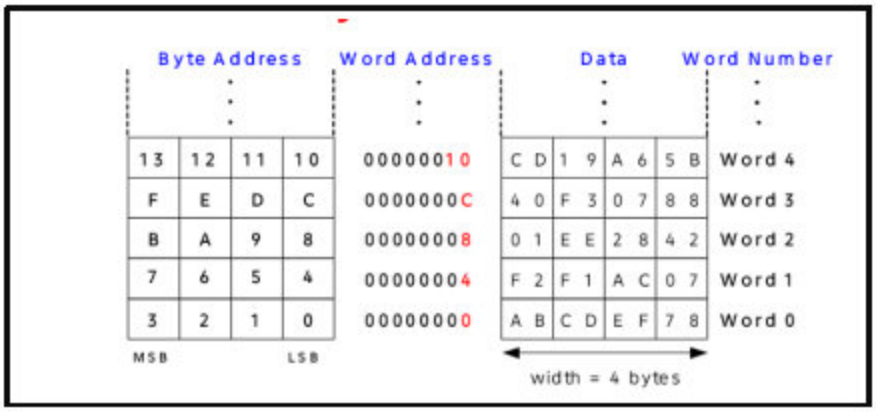
تسمى عملية الكتابة من الذاكرة بالتخزين *store word(sw)*

التعليمة:

```
swt4,0x3(zero)
#write the value in t4 #to memory word3
```

- العنونة بالبايت *Byte addressable*:

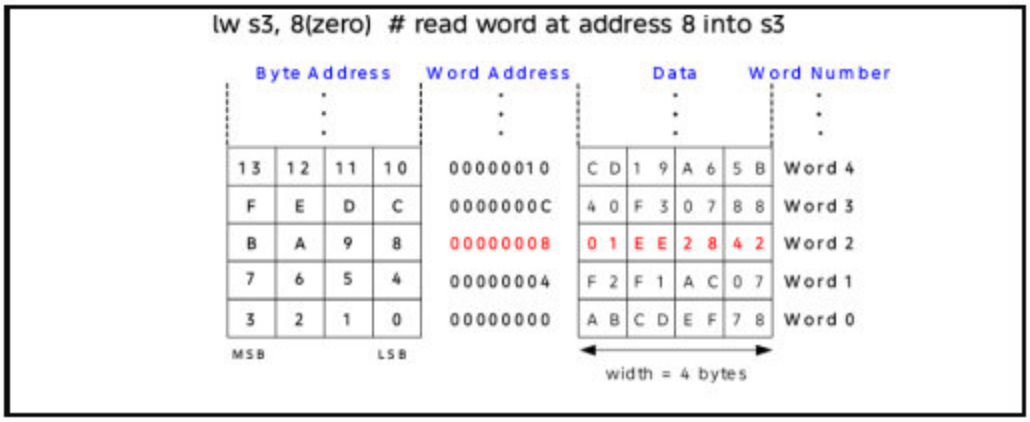
يتم عنونة كل كلمة 32 بت في 4 بايت متتالية في الذاكرة أي للحصول على عنوان كلمة معينة نضرب رقم الكلمة ب 4((بدنا الكلمة الثالثة مثلا ضربنا ب 4 طلع العنوان C بالسداسي عشر)) .



يتم عنونة *low byte* بال *low address* ((ب اول سطر اول بايت عاليمين عنوانه صفر والبايت الثاني عاليمين عنوانه واحد منكمل العد لنوصل للبايت الأول عاليمين بالكلمة الثانية بكون عنوانها 4)).

ملاحظة: معمارية RISC_V تعتمد على العنونة بالبايت

القراءة بالذاكرة ذات العنونة بالبايت:



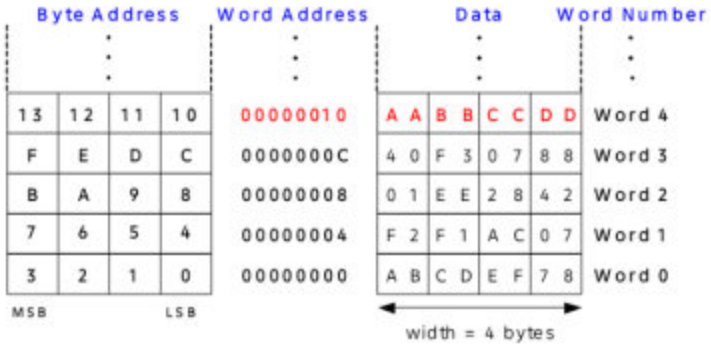
التعليمة:

```
lw s3, 8(zero)
 # read word at address 8 into s3
```

المعنى: قراءة الكلمة الثانية الموجودة في مسجل S3 ذات العنوان رقم 8

التخزين بالذاكرة ذات العنونة بالبايت(الكتابة):

sw t7, 0x10(zero) # write t7 into address 16



الموقع الثالث: العدد الثابت

تخزين العدد الثابت ب 12 بت

✓ التعليمة بلغة السي:

```
// int is a 32-bit signed word
int a =-372;
 int b = a + 6
```

✓ التعليمة بلغة التجميع:

```
# s0 = a, s1 = b
addi s0, zero,-372
addi s1, s0, 6
```

يتم تخزين العدد 372- في المسجل *S0* عن طريق جمع العدد الثابت مع المسجل الصفري ومن ثم جمع العدد 372- مع الرقم 6 وتخزين النتيجة في المسجل *S1*.

نعلم أن طول التعليمة 32 بت يتم حجز عشرين بت من أجل العملية والمسجل الهدف *S0* والمسجل المصدري *S1* ليتبقى لدينا 12 بت للعدد الثابت وبما أن هذا العدد عدد سالب يتم تكرار قيمة 1 حسب قاعدة تمديد الإشارة(*Sign Extension*) ، (لان القيمة سالبة ولو كانت موجبة بكون صفر)للعشرين بت المتبقية .

ملاحظة: بما أن المساحة المتاحة للرقم الثابت هي 12 بت فقط، وبما أنه رقم "ذو إشارة(Signed) "، أي يمكن أن يكون موجباً أو سالباً، فإن نطاق الأعداد التي يمكن للعتاد استيعابها محكوم رياضياً بالمعادلة التالية:

$$Range = [-2^{11} \dots 2^{11} - 1] = [-2048 \dots 2047]$$

ولكن!! في المثال :الرقم هو *-327*.وبما أن هذا الرقم يقع بأمان داخل النطاق (بين -2048 و 2047)، فإن المعالج يستطيع حقنه مباشرة داخل الـ12 بت المخصصة له في التعليمة. اي لا يمكن استخدام أكثر من 12 بت في هذه الطريقة

تخزين العدد الثابت ب32 بت

ومن أجل حل المشكلة نستخدم تعليمتي *lui* (تحميل الجزء العلوي الفوري) وتعليمة *addi*

✓ التعليمة بلغة سي:

```
int a = 0xFEDC8765;
```

✓ الكود بلغة التجميع:

```
# s0 = a
lui s0, 0xFEDC8
addi s0, s0,
```

تستطيع تعليمة *lui (Load upper immediat)* وضع العدد الثابت في عشرين بت عن طريق أخذ القسم العلوي فقط من ويتم تصفير القسم السفلي العدد *a* وال 12 بت المتبقية تصبح أصفار.

تجمع تعليمة *addi* المحتوى التي وضعناه في مسجل *s0* مع القسم السفلي من العدد السداسي عشر وبهي الطريقة وضعنا العدد بشكل كامل داخل المسجل *s0*.

C Code

```
int a = 0xFEDC8EAB;
```

Note: -341 = 0xEAB

RISC-V assembly code

```
# s0 = a
lui  s0, 0xFEDC9 # s0 = 0xFEDC9000
addi s0, s0, -341 # s0 = 0xFEDC9000 + 0xFFFFFEAB
                  #    = 0xFEDC8EAB
```

ملاحظة هامة جداً: في تعليمة addi دائما يتم التعامل مع ال 12 بت كأنها أعداد ذات إشارة في المثال أعلاه، الرقم هو 0x765. لو حولنا الخانة الأولى 7 إلى ثنائي ستكون البت الأخير هنا هو 0111. (أي أن الرقم يعتبره العتاد موجباً)، فلن تحدث مشكلة؛ لأن تمديد الإشارة سيضيف أصفاراً على اليسار قبل الجمع فلا تتأثر الـ 20 بت العلوية.

- **لكن ماذا لو كان الجزء السفلي يبدأ برقم سالب؟**

تخيل أن الجزء السفلي للرقم الذي تريده هو 0xEAB. الرقم E بالثنائي هو 1110. البت الأخير هنا هو 1! عندما ينفذ المعالج تعليمة addi s0, s0, 0xEAB سيعتبر الرقم سالباً، ويقوم العتاد تلقائياً بـ تمديد الإشارة عبر وضع وحدات (1s) على يسار الرقم لتصبح القيمة المضافة فعلياً هي 0xFFFFFEAB. عند جمع 0xFFFFFEAB مع القيمة العلوية، ستتخرب الـ 20 بت العلوية.

ولكن في الوقت ذاته نلاحظ أنه قد تم طرح واحد من القسم العلوي ليصبح FEDC7 بدلاً من FEDC8.

لحل هذه المشكلة يقوم المترجم (Compiler) بخدعة استباقية: بما أننا نعلم يقيناً أن تعليمة addi ستطرح 1 من الجزء العلوي نتيجة الـ 1 المتواجد في البت 11، فإننا نقوم بإضافة 1 مسبقاً إلى الجزء العلوي داخل تعليمة lui. كما هو موضح في الرقم الذي لونه أحمر في الصورة.

البوابات المنطقية

بوابة *AND*:

تستخدم بشكل عام لحجب البتات غير المرغوبة *mascking* عند ضربها بصفر.

مثال لتوضيح الفكرة: عندما نريد كتابة الرقم 1 من الكيبورد ورمز الاسكي للرقم ذ هو 0031 من أجل التخلص من الرقم 3 وإظهار الواحد على الشاشة يتم تنفيذ عملية *AND* على كود الاسكي مع 0001

مثال من السلايدات:

0xF234012F AND 0x000000FF = 0x0000002F

بوابة *OR*:

تستخدم بشكل عام لدمج بتات المسجلات مع بعضهما.

مثال من السلايدات:

0xF2340000 OR 0x000012BC = 0xF23412BC

بوابة *XOR*:

في حال الاختلاف يعطي 1 وفي حال التشابه تعطي 0).

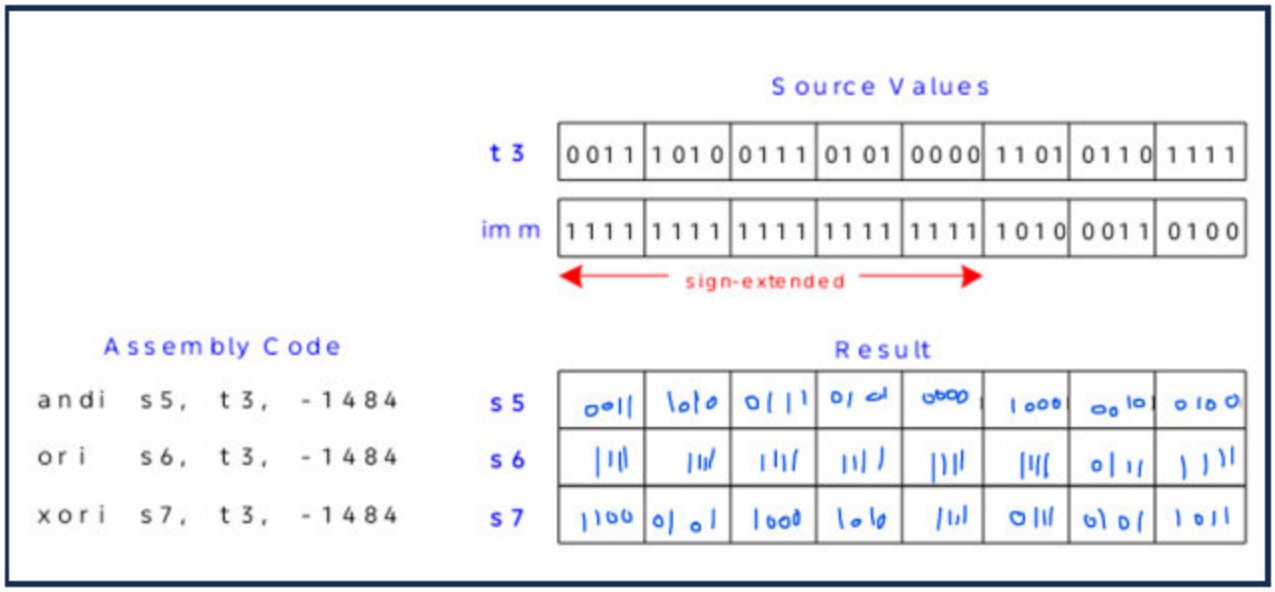
ملاحظة من السلايدات: من الممكن لبوابة *XOR* أن تكافئ عاكس إذا كان:

- A XOR -1 = NOT A (remember that -1 = 0xFFFFFFFF)

توضيح: إذا اردنا عكس قيمة كاملة نقوم بقلب البتات عبر حيلة هندسية باستخدام تعليمة *xor* مع الرقم -1. الرقم -1 في نظام المتمم الثنائي يتم تمثيله بـ 32 بت كلها وحدات (*xFFFFFFFF*0).عندما نمرر أي مسجل *A* ونعمل له *xor* مع مسجل مليء بالوحدات، فإن البوابات الإلكترونية ستقوم بقلب كل بت من بتات *A*

$$A \oplus 0xFFFFFFFF = NOT\ A$$

مثال شامل من السلايدات:



- ✓ أخذنا المتمم الثنائي ل 1484 لأنه سالب سنحصل على *A34* في السداسي عشر قمنا بتحويله إلى الثنائي وأضفنا واحدات في البتات المتبقية حسب قاعدة تمديد الإشارة(واحدات لأن العدد سالب)
- ✓ طبقنا العمليات الثلاث على المسجل *t3* والعدد الثابت *imm* لنحصل على النتائج كما في الشكل

ملاحظة: andi ,ori تعني عملية منطقية مع عدد ثابت

تعليمات الإزاحة

تخضع الإزاحة إلى قاعدة الخمس بتات لأن عدد البتات الكلي هو 32 بت إذا قمنا بالإزاحة أكثر من 32 خطوة ستكون البتات داخل المسجل إما جميعا أصفار أو جميعها واحدات بحسب نوع الإزاحة لذلك يكون أقصى حد منطقي للإزاحة هو 31.

أنواع الإزاحة:

- ***Sll(shift left logic):***

عندما نزيح مرتبة وندخل مكانها صفر تسمى إزاحة منطقية

```
sll t0, t1, t2
# t0 = t1 << t2
```

(كل عملية إزاحة تمثل عملية ضرب ب 2).

SRL(shift right logical):

```
srl t0, t1, t2
# t0 = t1 >> t2
```

(كل عملية إزاحة تمثل عملية قسمة على2)

ملاحظة: تستخدم الإزاحة المنطقية مع الأعداد غير المؤشرة وتستخدم الإزاحة الرياضية مع الأعداد المؤشرة

SRA(shift right arithmetic):

إذا كان الرقم موجباً (يبدأ بـ 0)، تدخل أصفار. وإذا كان الرقم سالباً (يبدأ بـ 1)، تدخل واحدات (1) للحفاظ على قيمة الإشارة السالبة.

المعنى الرياضي: تعادل القسمة الحسابية على 2^n مع المحافظة على الإشارة السالبة للرقم.

```
sra t0, t1, t2
# t0 = t1 >>> t2
```

تعليمات الإزاحة مع عدد ثابت:

- slli: shift left logical immediate
  - Example: slli t0, t1, 23 # t0 = t1 << 23
- srli: shift right logical immediate
  - Example: srli t0, t1, 18 # t0 = t1 >> 18
- srai: shift right arithmetic immediate
  - Example: srai t0, t1, 5 # t0 = t1 >>> 5

تعليمات الضرب والقسمة

الضرب:

عند ضرب رقمين، كل منهما يتكون من 32 بت، فإن الناتج كحد أقصى قد يحتاج إلى 64بت ليتسع له بالكامل دون حدوث طفح رقمي.(Overflow)

بما أن مسجلات المعالج (Registers) في هذا النظام حجمها 32 بت فقط، لا يمكن للمعالج وضع ناتج الـ 64 بت في مسجل واحد. لذلك، يقوم المعالج بتقسيم الناتج إلى جزأين ويستخدم أمرين منفصلين:

- ✓ *mul (Multiply)* يحسب عملية الضرب ويأخذ النصف السفلي فقط (أول 32 بت من اليمين).
- ✓ *mulh (Multiply High Signed)*: يحسب نفس عملية الضرب ولكن يأخذ النصف العلوي فقط (آخر 32 بت من اليسار)، مع الأخذ في الاعتبار أن الأرقام بها إشارة (موجب/سالب)

بدمج المسجلين معاً *{s4, s3}*، نحصل على النتيجة الكلية الصحيحة.

مثال من السلايدات:

Example: s1 = 0x40000000 = 2^{30}; s2 = 0x80000000 = -2^{31}

s1 x s2 = -2^{61} = 0xE0000000 00000000

s4 = 0xE0000000; s3 = 0x00000000

القسمة:

في معالجات *RISC* لا تعود عملية القسمة بـ "خارج القسمة" و"الباقي" في أمر *(Instruction)* واحد، بل يتم تقسيمها إلى أمرين منفصلين لتسهيل تصميم وحدة الحساب والمنطق *(ALU)* وتسريع الأداء.

```
div s3, s1, s2 # s3 = s1/s2
rem s4, s1, s2 # s4 = s1%s2
```

- تعليمة *div* (للناتج) تقسم العددين وتخزن ناتج القسمة في *s1*
- تعليمة *rem* (للباقي) تقسم العددين نفسهما وتخزن الباقي في *s4*

مثال من السلايدات:

Example: s1 = 0x00000011 = 17; s2 = 0x00000003 = 3

s1 / s2 = 5

s1 % s2 = 2

s3 = 0x00000005; s4 = 0x00000002

تعليمات التفرع والقفز branches and jumps

في الحالة العادية، يقوم المعالج بتنفيذ الأوامر برمجياً بشكل تتابعي خلف بعضها البعض؛ حيث يزداد "عداد البرنامج *(Program Counter - PC)*" تلقائياً بمقدار 4 بايت للانتقال للأمر التالي. لكن البرمجة تحتاج إلى اتخاذ قرارات مثل *if-else* أو تكرار عمليات مثل *Loops* هنا يأتي دور التفرع والقفز لتعديل قيمة الـ*PC* وجعل المعالج "يقفز" إلى سطر برمي آخر خارج الترتيب.

أنواع التفرع *(Types of branches)*:

- مشروط (بناءاً على نتيجة) *(Conditional)*
  - التفرع في حالة التساوي *(beq - branch if equal)*
  - التفرع في حالة عدم التساوي *(bne - branch if not equal)*
  - التفرع إذا كان أقل من *(blt - branch if less than)*
  - التفرع إذا كان أكبر من أو يساوي *(bge - branch if greater than or equal)*

مثال:

```
addi s0, zero, 4        # s0 = 0 + 4 = 4
addi s1, zero, 1        # s1 = 0 + 1 = 1
slli s1, s1, 2     # s1 = 1 << 2 = 4
beq  s0, s1, target     # branch is taken
addi s1, s1, 1        # not executed
sub  s1, s1, s0         # not executed

target:                 # label
add  s1, s1, s0         # s1 = 4 + 4 = 8
```

الشرح: وضعنا قيمة 4 في المسجل *s0* ووضعنا قيمة 1 في المسجل *s1* قمنا بإزاحة لليسار بمقدار خانتين (ضربنا ب 4) فحصلنا على 4 في المسجل *s1* تعليمة *beq* تقارن اذا كان محتوى المسجلين متساويين وفي حال التساوي (وهي حالة محققة) سيتجاوز جميع التعليمات التالية ليذهب مباشرة إلى العنوان *label* الموجود في التعليمة (*target*) وهنا يظهر مفهوم التفرع ((تنفيذ غير متتابع للتعليمات))

ملاحظة: العناوين :(Labels) تُشير إلى موقع تعليمة (أمر) في الذاكرة. لا يمكن أن تكون كلمات محجوزة بالنظام، ويجب أن تنتهي بنقطتين .(:)

مثال2:

```
addi  s0, zero, 4        # s0 = 0 + 4 = 4
addi  s1, zero, 1        # s1 = 0 + 1 = 1
slli   s1, s1, 2         # s1 = 1 << 2 = 4
bne          s0, s1, target     # branch not taken
addi  s1, s1, 1          # s1 = 4 + 1 = 5
sub   s1, s1, s0         # s1 = 5 – 4 = 1

target:
add   s1, s1, s0         # s1 = 1 + 4 = 5
```

استخدمنا تعليمة في حال عدم التساوي ولكن المسجلين متساويين وبذلك الشرط غير محقق وسيعمل البرنامج بشكل تتابعي دون أن يتأثر بتعليمة التفرع

- **غير مشروط (غير مقترن بنتيجة)** ***(Unconditional)***:
  - **القفز** ***(j - jump)***
  - **القفز إلى مسجل** ***(jr - jump register)***
  - **القفز والربط** ***(jal - jump and link)***
  - **القفز والربط بموجب مسجل** ***(jalr - jump and link register)***

مثال على القفز الغير مشروط:

```
j        target        # jump to target
srai     s1, s1, 2          # not executed
addi     s1, s1, 1          # not executed
sub      s1, s1, s0    # not executed

target:
add          s1, s1, s0         # s1 = 1 + 4 = 5
```

كتبنا تعليمة القفز إلى target فقام بشكل مباشر بتجاوز جميع التعليمات التي قبل العنوان target لتنفيذها.

الشروط والحلقات التكرارية

If statement:

في لغات البرمجة العالية المستوى يتم فحص تحقق الشرط إذا كان محقق تتنفذ التعليمة التي داخل بلوك الشرط وبعدها التعليمة الأخرى ولكن في لغة التجميع من الأسهل أن نفحص عدم تحقق الشرط حيث في المثال نقول إذا كانت *i* لا تساوي *j* ((أي نتحقق من عدم تساوي يمكننا استخدام تعليمة *bne*)) سينقل إلى التعليمة التي خارج بلوك الشرط أما إذا كانا متساويين سيعمل البرنامج بشكل متتابع دون أي قفز أو تجاوز لأي تعليمة.

C Code

```
if (i == j)
  f = g + h;

else
  f = f – i;
```

RISC-V assembly code

```
# s0 = f, s1 = g, s2 = h
# s3 = i, s4 = j
```

إذا أردنا لتعليمة واحدة فقط أن تتنفذ نستعمل:

If else statement:

في لغات البرمجة عالية المستوى لا يمكن تنفيذ ماداخل بلوك *if* وما داخل بلوك *else* معاً تحقيق ذلك بكفاءة نستخدم:

C Code

```
if (i == j)
  f = g + h;

else
  f = f – i;
```

RISC-V assembly code

```
# s0 = f, s1 = g, s2 = h
# s3 = i, s4 = j
```

1. أمر تفرع مشروط *(bne)*: لاختيار المسار المناسب في البداية.
2. أمر قفز غير مشروط *(j)*: وضعه في نهاية مسار ال *if* لإجبار المعالج على "القفز فوق" مسار ال *else* لمنع تداخل العمليات.

لـ شرح كيف يتم هذا نميز حالتين:

حالة تحقق الشرط *i=j* :

نفحص التعليمة الأولى *bne s3, s4, Else* وبما أن الشرط محقق لن يتم التفرع أو تجاوز أي تعليمة داخل بلوك *if* ومنه ستتم عملية $f = g + h$ ثم عن طريق تعليمة *j exit* وهو قفز غير مشروط لضمان عدم الدخول إلى بلوك *else* ليتم إنهاء البرنامج.

في حال عدم تحقق الشرط *i!=j* :

نفحص التعليمة الأولى *bne s3, s4, Else* وبما أن الشرط غير محقق يتم التفرع أو تجاوز التعليمة داخل بلوك *if* لتنفيذ التعليمة التي بداخل بلوك *else* منه ستتم عملية $f = i - f$ ثم يكمل البرنامج تتابعه بشكل طبيعي ليصل إلى تعليمة الخروج *Exit* لإنهاء البرنامج.

حلقة *while*:

في اللغات عالية المستوى، تستمر حلقة *while* في الدوران طالما أن الشرط محقق *(pow != 128)*. ولغة التجميع تختبر الحالة العكسية *(pow == 128)* للشرط الموجود في اللغة عالية المستوى *(pow != 128)*

C Code

```
// determines the power
// of x such that 2^x = 128
int pow = 1;
int x   = 0;

while (pow != 128) {
  pow = pow * 2;
  x = x + 1;
}
```

RISC-V assembly code

```
# s0 = pow, s1 = x
```

حيث سنستخدم أمر التفرع في حالة التساوي beq (Branch If Equal) إذا تساوت القيمة مع 128، يقفز المعالج خارجاً وينتهي التكرار. وإذا لم تتساوَ يستمر داخل جسم الحلقة ونستخدم تعليمة الإزاحة لليسار لتمثيل عملية الضرب و تعليمة addi لعملية الجمع ثم نضع أمر قفز غير مشروط while test j في النهاية ليعود بالمعالج إلى بداية الفحص مجدداً وعند تحقق شرط الحلقة يتم تحقق الشرط العكسي بنجاح يقفز المعالج مباشرة متخطياً جسم الحلقة وأمر العودة، ويهبط عند العنوان :.While_exit عند هذه النقطة، يتوقف التكرار وتكون النتيجة النهائية المستقرة في المسجلات هي $x = 7$:لأن $2^7 = 128$.

حلقة *for*:

في اللغات عالية المستوى مثل *C* أو *java* تُكتب حلقة *for* في سطر واحد يجمع كل عناصر التحكم بالتكرار تسهيلاً على المبرمج. لكن المعالج *(CPU)* لا يمكنه هندسياً فهم هذا السطر المركب دفعة واحدة. لذلك يقوم المترجم *(Compiler)* بفك هذه الحلقة وتحويلها إلى بنية تشبه تماماً حلقة الـ *while* التي شرحناها سابقاً، مع توزيع المكونات الأربعة في أسلوب تتابعي منظم.

C Code

```
// add the numbers from 0 to 9
int sum = 0;
int i;

for (i=0; i!=10; i = i+1) {
  sum = sum + i;
}
```

RISC-V assembly code

```
# s0 = i, s1 = sum
```

آلية عمل الكود في لغة التجميع:

1- نضع قيمة المتغير *sum* في المسجل *s1* وقيمة متغير الحلقة *i* في المتغير *s0* عن طريق تعليمة *add*

2- نضع عنوان لبداية الحلقة *for test:* لتتم مرحلة فحص الشرط العكسي وضع القيمة الثابتة 10 في مسجل مؤقت *t0* لإجراء المقارنة

3- التحقق من الشرط العكسي عبر تعليمة *beq s0, t0, FOR_EXIT* اذا كان *i=1* يتحقق الشرط ونذهب إلى نهاية الحلقة وعند عدم تحقق الشرط سيتم الدخول إلى جسم الحلقة وتنفيذ التعليمة *add s1, s1, s0 # sum = sum + i*

4- من أجل تحديث قيمة متغير الحلقة عبر التعليمة *addi s0, s0, 1 # i = i + 1* ونعود إلى العنوان *for test:* لإعادة الفحص من جديد.

Less than comparsion

C Code

```
// add the powers of 2 from 1
// to 100
int sum = 0;
int i;

for (i=1; i < 101; i = i*2) {
  sum = sum + i;
}
```

RISC-V assembly code

```
# s0 = i, s1 = sum
      addi  s1, zero, 0
      addi  s0, zero, 1
      addi  t0, zero, 101
loop:
      slt   t2, s0, t0
      beq   t2, zero, done
      add   s1, s1, s0
      slli  s0, s0, 1
      j     loop
done:
```

slt: set if less than instruction
slt t2, s0, t0 # if s0 < t0, t2 = 1
                #
otherwise t2 = 0

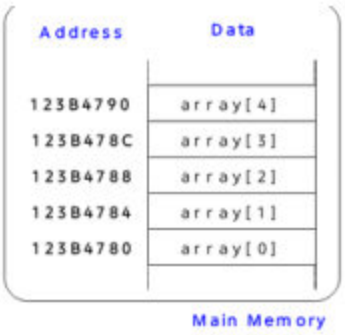
آلية عمل الكود في لغة التجميع:ف2

1- تهيئة المسجلات حيث *S0=i,S1=Sum* عبر تعليمة *addi* ولإسناد القيم الأولية
2- ندخل إلى عنوان الحلقة وننتبه إلى التعليمة الجديدة *slt(set less than)* تأخذ *t2* قيمة 1 إذا كان *s0* أصغر من*t0* وبخلاق ذلك تكون *t2* تساوي و يتم وضع داخل عنوان الحلقة الشرط العكسي لكسر الحلقة *bge s0, t0, EXIT* (إذا كانت*i >= 101* ، اخرج من الحلقة وانتقل إلى*EXIT*).
3- في حال تحقق الشرط العكسي يتم الخروج من الحلقة وفي حال عدم تحقق الشرط يتم تنفيذ التعليمات في جسم الحلقة من أجل زيادة المجموع *add s1, s1, s0*(*# sum = sum + i*) ومن أجل زيادة عداد الحلقة عن طريق الضرب ب 2 نستخدم تعليمة الإزاحة *slli s0, s0, 1* ومن ثم نعود بقفز غير مشروط إلى بداية الحلقة.

المصفوفاتarrays

```
// C Code
    int array[5];
    array[0] = array[0] * 2;
    array[1] = array[1] * 2;

# RISC-V assembly code
# s0 = array base address
```



آلية عمل الكود في لغة التجميع:

1. التحميل *(Load - lw)*: جلب البيانات من عنوان الذاكرة البعيد ووضعها في السجلات *(Registers)* مثل *t0* و *t1).*

- *lw t0, 0(s0)* تعني: اذهب إلى العنوان المخزن في *s0* بـ إزاحة *(Offset)* مقدارها *0* بايت، واقرأ الـ الكلمة *(Word = 4 Bytes)* وضعها في *t0.*

- *lw t1, 4(s0)* تعني: اذهب إلى العنوان *s0* وتحرّك بمقدار *4* بايت للأمام للوصول إلى العنصر الثاني، ثم اقرأه وضعه في *t1.*

2. المعالجة *(Process - slli)*: إجراء العملية الحسابية داخل وحدة الحساب والمنطق*.(ALU)*

- هندسياً، إزاحة النظام الثنائي *(Binary)* خطوة واحدة جهة اليسار تضاعف القيمة تماماً

3. **التخزين (Store - sw):** إعادة النتيجة المحسوبة من السجل السريع إلى مكانها الدائم في الذاكرة العشوائية.

   - `sw t0, 0(s0)` تأخذ النتيجة من `t0` وتكتبها فوق القيمة القديمة لـ `array[0]`.

مثال:

بفرض لدينا مصفوفة تحوي 1000 عنصر ونريد ضرب كل عنصر من عناصر المصفوفة ب 8:

```
# RISC-V assembly code
# s0 = array base address, s1 = i
# initialization code
  lui  s0, 0x23B8F        # s0 = 0x23B8F000
  ori  s0, s0, 0x400      # s0 = 0x23B8F400
  addi s1, zero, 0        # i = 0
  addi t2, zero, 1000     # t2 = 1000

loop:
  bge  s1, t2, done       # if not then done
  slli t0, s1, 2          # t0 = i * 4 (byte offset)
  add  t0, t0, s0         # address of array[i]
  lw   t1, 0(t0)          # t1 = array[i]
  slli t1, t1, 3          # t1 = array[i] * 8
  sw   t1, 0(t0)          # array[i] = array[i] * 8
  addi s1, s1, 1          # i = i + 1
  j    loop               # repeat
done:
```

سنشرح هذا الكود بالتفصيل:

- تحميل القيمة `23B8F` ووضعها في مسجل `S0` (في أعلى 20 بت)
- يتم دمج القيمة في `S0` مع 400 (12 بت) عن طريق بوابة `OR` ووضع الناتج في `S0`
- تهيئة عداد الحلقة
- وضع قيمة متغير الحلقة 1000 في مسجل مؤقت
- نكتب عنوان الحلقة `LOOP:`
- نكتب الشرط العكسي لكسر الحلقة
- نضرب `S1` ب 4 (إزاحة مرتين) ونضع النتيجة في `t0` (هذا يمثل الازاحة بمقدار 4 بين كل عنوان)
- `add t0, t0, s0` يجمع الإزاحة التي حسبناها للتو `(t0)` مع العنوان الأساسي للمصفوفة `(s0)`. الآن أصبح السجل `t0` يحتوي على العنوان الدقيق والمباشر للعنصر `array[i]` داخل الذاكرة العشوائية.
- `lw t1, 0(t0)`: جلب القيمة من العنوان المحسوب وتخزينها مؤقتاً في `t1`.

- **`slli t1, t1, 3`: ضرب القيمة في 8 بإزاحتها 3 بتات إلى اليسار. *($2^3 = 8$)* وهي طريقة أسرع بكثير من استخدام وحدة الضرب *(Multiplier)* في المعالج.**
- **`sw t1, 0(t0)`: إعادة تخزين النتيجة المضروبة في 8 في نفس مكانها بالذاكرة.**
- **زيادة عداد الحلقة**
- **قفز مباشر إلى بداية الحلقة**
- **انهاء البرنامج عند تحقق الشرط العكسي**

جدول ASCII

#	Char	#	Char	#	Char	#	Char	#	Char	#	Char
20	space	30	0	40	@	50	P	60	`	70	p
21	!	31	1	41	A	51	Q	61	a	71	q
22	"	32	2	42	B	52	R	62	b	72	r
23	#	33	3	43	C	53	S	63	c	73	s
24	$	34	4	44	D	54	T	64	d	74	t
25	%	35	5	45	E	55	U	65	e	75	u
26	&	36	6	46	F	56	V	66	f	76	v
27	'	37	7	47	G	57	W	67	g	77	w
28	(	38	8	48	H	58	X	68	h	78	x
29	)	39	9	49	I	59	Y	69	i	79	y
2A	*	3A	:	4A	J	5A	Z	6A	j	7A	z
2B	+	3B	;	4B	K	5B	[	6B	k	7B	{
2C	,	3C	<	4C	L	5C	\	6C	l	7C	|
2D	-	3D	=	4D	M	5D	]	6D	m	7D	}
2E	.	3E	>	4E	N	5E	^	6E	n	7E	~
2F	/	3F	?	4F	O	5F	_	6F	o		

استدعاء الدوال Function calles

- ***Caller*: الذي يستدعي**
- ***Calle*: المستدعى**

C Code

```
int main() {
  simple();
  a = b + c;
}

void simple() {
  return;
}
```

RISC-V assembly code

```
0x00000300 main:  jal  simple     # call
0x00000304        add  s0, s1, s2
...               ...

0x0000051c simple: jr  ra         # return
```

void means that simple doesn't return a value

jal simple:

ra = PC + 4 (0x00000304)

jumps to simple label (PC = 0x0000051c)

jr ra:

PC = ra (0x00000304)

- عند الوصول إلى تعليمة استدعاء الدالة يتم الذهاب إلى داخل الدالة المستدعاة مع الحفاظ على العنوان التي توقف عنده عبر تعليمة *jump and link(jal)*
- قفز: *(jump)* يقوم المعالج بتغيير قيمة عداد البرنامج *PC* ليصبح مساوياً لعنوان الدالة *simple*
- الربط *(link)*: بعد إنهاء الدالة *simple* يجب أن يعود لتنفيذ السطر الذي يليه *(a = b + c)*. هذا السطر يقع عند العنوان التالي مباشرة
- عند الدخول إلى الدالة والوصول إلى *return* أي نعود إلى البرنامج ويتم هذا في لغة التجميع عن طريق تعليمة *jump register (jr)* التي تعمل على الشكل التالي:
- معالج يقرأ القيمة المخزنة داخل المسجل *(ra* والتي وضعناها سابقاً في خطوة الاستدعاء وكانت *0x00000304).*
- يقوم بوضع هذه القيمة مباشرة داخل عداد البرنامج *PC (PC = ra).*

النتيجة: المعالج يعود تلقائياً إلى الدالة الرئيسية *main* وتحديداً إلى السطر التالي لاستدعاء الدالة ليتنفذ كود الأسمبلي المترجم للعملية الحسابية:

مثال تطبيقي:

C Code

```
nt main()
{
 int y;
 ...
 y = diffofsums(2, 3, 4, 5);  // 4 arguments
 ...
}

nt diffofsums(int f, int g, int h, int i)
{
 int result;
 result = (f + g) - (h + i);
 return result;              // return value
}
```

RISC-V assembly code

```
# s7 = y
main:
...
addi a0, zero, 2  # argument 0 = 2
addi a1, zero, 3  # argument 1 = 3
addi a2, zero, 4  # argument 2 = 4
addi a3, zero, 5  # argument 3 = 5
jal  diffofsums   # call function
add  s7, a0, zero # y = returned value
...
# s3 = result
diffofsums:
add  t0, a0, a1   # t0 = f + g
add  t1, a2, a3   # t1 = h + i
sub  s3, t0, t1   # result = (f + g) - (h + i)
add  a0, s3, zero # put return value in a0
jr   ra           # return to caller
```

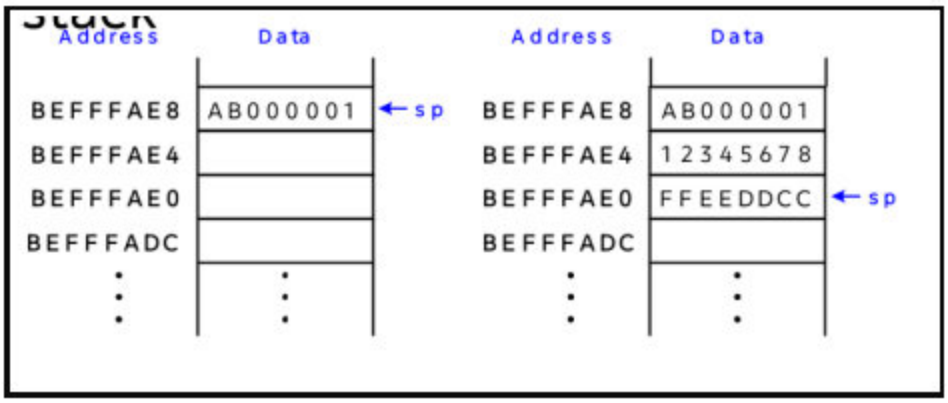
✓ التعليمات باللون الأزرق لتهيئة المسجلات ومن ثم انتقلنا إلى عنوان الدالة *diffosums:* التي تحوي بداخلها قيمة النتيجة التي ظهرت من خلال عدة تعليمات جمع (نلاحظ داخل الدالة استخدما للتخزين مسجلات مؤقتة لأن العملية كانت على أكثر من مرحلة وفي اخر مرحلة خزنا بمسجل *s*) .

ملاحظة: لو ان المسجلات التي استخدمناها داخل الدالة نستخدمها في البرنامج الأساسي لكان يجب علينا حفظ المعلومات في هذه المسجلات في مكدسات إلى حين استدعاء الدالة.

المكدس stack

يتم تعبئته باتجاه تناقص العناوين.

Sp يدل على عنوان بداية المكدس



استخدام المكدس:

```
# RISC-V assembly
# s3 = result
diffofsums:
  add  t0, a0, a1   # t0 = f + g
  add  t1, a2, a3   # t1 = h + i
  sub  s3, t0, t1   # result = (f + g) – (h + i)
  add  a0, s3, zero # put return value in a0
  jr   ra           # return to caller
```

لو أن الدالة الرئيسية main كانت تستخدم المسجل s3 لتخزين قيمة مهمة جداً (مثل رصيد حساب بنكي، أو عداد حلقة تكرارية). عندما قامت main باستدعاء الدالة diffofsums، قامت الأخيرة باستخدام نفس المسجل s3لتخزين ناتج الطرح الخاص بها بدون علم الدالة الرئيسية عندما يعود المعالج إلى الدالة main، سيجد أن القيمة القديمة والمهمة التي كانت في s3قد مُسحت وتغيرت بقيمة جديدة تماماً نتجت عن الدالة الفرعية. هذا هو "الأثر الجانبي غير المقصود" الذي يتسبب في انهيار البرامج.

ومن أجل تفادي هذه المشكلة يفضل استخدام المسجلات المؤقتة لحفظ معلومات المكدس ومسجلات s لحفظ معلومات الدالة الرئيسية.

مثال تطبيقي:

```
# s3 = result
diffofsums:
  addi sp, sp, -12      # make space on stack to
                        # store three registers
  sw   s3, 8(sp)        # save s3 on stack
  sw   t0, 4(sp)        # save t0 on stack
  sw   t1, 0(sp)        # save t1 on stack
  add  t0, a0, a1       # t0 = f + g
  add  t1, a2, a3       # t1 = h + i
  sub  s3, t0, t1       # result = (f + g) - (h + i)
  add  a0, s3, zero     # put return value in a0
  lw   s3, 8(sp)        # restore s3 from stack
  lw   t0, 4(sp)        # restore t0 from stack
  lw   t1, 0(sp)        # restore t1 from stack
  addi sp, sp, 12       # deallocate stack space
  jr   ra               # return to caller
```

ملاحظات على الكود:

((حطينا ب اول تعليمة 12- لان 3 مسجلات بتاخد تلت عناوين بالذاكرة المؤقتة وبين كل عنوان وعنوان 4))

- بما أن الدالة *diffofsums* مجبرة على استخدام المسجلات *s3* و *t0* و *t1* لإجراء حساباتها، وبما أنها لا تريد مسح بيانات الدالة الرئيسية *(main)* ، فإنها تقوم بعمل "نسخة احتياطية" من قيم هذه المسجلات الحالية وتضعها في الذاكرة (ال *Stack)* قبل أن تبدأ بأي شيء:

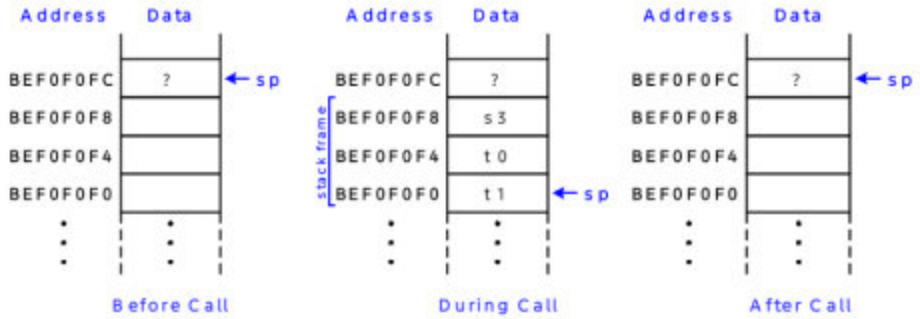
1. تقوم بتوسيع ال *Stack* بمقدار 12 بايت عن طريق سحب مؤشر المكدس *sp* لأسفل *(addi sp, sp, -12).*
2. تقوم بالتقاط القيم الحالية للمسجلات ونسخها في تلك المساحة المحجوزة عبر تعليمة التخزين في الذاكرة *sw.*

- قمنا بحفظ المسجلات *s1,t3, t0* في المكدس عبر تعليمة *sw*
- يتم استدعاء القيم الثلاث عن طريق تعليمة *lw*

addi sp, sp, 12 # deallocate stack space:

- إضافة 12 إلى مؤشر المكدس *sp* لإلغاء حجز المساحة وإرجاع المؤشر إلى وضعه الأصلي قبل الخروج.

أمثلة أخرى:

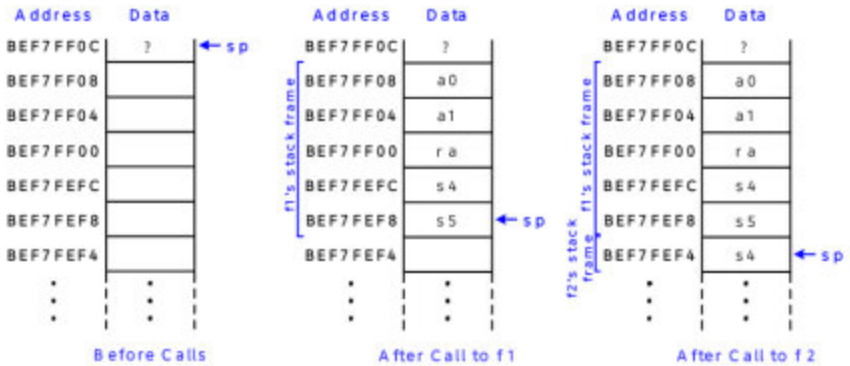



```
addi sp, sp, -4      # make space on stack to
                     # store one register
sw   s3, 0(sp)       # save s3 on stack
add  t0, a0, a1      # t0 = f + g
add  t1, a2, a3      # t1 = h + i
sub  s3, t0, t1      # result = (f + g) – (h + i)
add  a0, s3, zero    # put return value in a0
lw   s3, 0(sp)       # restore $s3 from stack
addi sp, sp, 4       # deallocate stack space
jr   ra              # return to caller
```

Non leaf function:

Non-leaf function:
a function that calls another function

```
func1:
  addi sp, sp, -4   # make space on stack
  sw   ra, 0(sp)    # save ra on stack
  jal  func2
  ...
  lw   ra, 0(sp)    # restore ra from stack
  addi sp, sp, 4    # deallocate stack space
  jr   ra           # return to caller
```



تتمة الفصل الخامس

Jumps

عندما نستدعي تابع ونذهب إلى التابع المستدعى يجب أن يتم حفظ عنوان تعليمة الاستدعاء لاستكمال عمل البرنامج ويتم هذا عن طريق تعليمات القفز الغير مشروط

لدينا نوعان من القفز الغير مشروط في معمارية *RISC_V*:

- *Jump and link (jal)*:

برامتراتها: $(rd, immed_{20:0})$

يأخذ المعالج العنوان التالي للتعليمية الحالية *(PC + 4)* ويقوم بتخزينه داخل السجل *rd.* هذا يضمن أنه عندما تنتهي الدالة، يستطيع المعالج العودة للمكان الصحيح.(حفظ خط الرجعة) ومنه:

$$rd = pc + 4$$

حساب عنوان القفز: يقوم المعالج بإضافة القيمة الفورية *(imm)* مباشرة إلى العداد الحالي *PC.* (حجمها 20 بت مما يسمح بتنقل بمدى قفز محدود) فيكون:

$$pc = pc + immed_{20:0}$$

- *Jump and link register*

برامتراتها: $(jalr\ rd\ ,rs, imme_{11:0})$

يتم حفظ عنوان العودة بشكل مماثل للتعليمة الأولى ولكن يكون الاختلاف في حساب عنوان القفز حيث يتم التخزين في سجل عداد البرنامج ما يلي:

$$pc = [rs] + signExt(imm)_{11:0}$$

التعليمات الزائفة Pseudoinstruction

هي تعليمات غير موجودة في المعالج ولكنها تكون مقروءة بشكل أوضح للمبرمج ويتم ترجمتها من تعليمات زائفة إلى تعليمات حقيقية داخل ال *assembler* (المجمع)

التعليمات الزائفة للقفز:

تحوي معمارية ريسك فايف على أربع تعليمات قفز زائفة:

التعليمةالأولى: تعليمة *j imm*

(*jal x0,imm*) وهي تعني اقفز واربط مع المسجل الصفري (تستخدم بكثرة في الحلقات التكرارية)

في هذه التعلمية لا يمكننا العودة إلى المكان الذي قفزنا منه والسبب أننا وضعنا مسجل صفري بدل من مسجل *ra* مهما وضعنا فيه عناوين سيتم تجاهلها.

التعليمة الثانية: تعليمة *jal imm*

أي $(jal\ ra, imm)$ وهي تستخدم عند استدعاء التوابع في الكود حيث يكتفي المبرمج بكتابة عنوان القفز ليفهم المجمع وحده أنه يجب عليه تخزين هذا العنوان في مسجل *return address*.

التعليمة الثالثة: تعليمة *jr rs*

أي $(jarl\ x0, rs, 0)$تستخدم للقفز إلى عنوان تم حفظه مسبقاً في مسجل وبما أننا لا نريد حفظ مكان العودة نجعل $rd = X0$ و$imm = 0$ لان العنوان موجود داخل *rs* دون الحاجة إلى إزاحته.

التعليمة الرابعة: تعليمة *jr ra*

$(jalr\ x0, ra, 0)$ أي *0* أو يمكننا كتابة *ret* : يتم استدخدامها للعودة إلى البرنامج الرئيسي بعد تنفيذ الدالة أي اذهب إلى العنوان الموجود في *ra*واجمع معه *0*، ولا تحفظ عنواناً للعودة لأننا نعود بالفعل *(x0)*.

ملاحظة: عند كتابة التعليمات الزائفة لا يشترط كتابة الوسطاء لأن المجمع سيفهم وحده التعليمة الزائفة مع وسطائها

العنوان Label

رأينا في المحاضرة السابقة أننا يمكننا كتابة العنوان عند بداية الدالة ولكن المجمع لا يفهم الكلمات وإنما يفهم العناوين

نستخدم طريقة تسمى العنونة النسبية لعداد البرنامج *(PC-Relative Addressing)*. بمعنى آخر، يسأل المعالج نفسه (كم بايت يبعد الهدف عن مكاني الحالي) هذه المسافة تسمى *Immediate Offset (imm)*.

مثال تطبيقي

0x00000300 main jal simple #call

0x00000304 add s0,s1,s1

... ...

0x0000051c simple: jr ra #return

بفرض أن تعليمة القفز موجودة في العنوان *0x00000300* والدالة *simple* تقع عند العنوان *0x0000051C*

يتم حساب الإزاحة المناسبة عبر القانون التالي:

$$Offset = 0x51C - 0x300 = 0x21C$$

ويتم هذا الأمر برمجياً ليصبح لدينا التعليمة $jal\ ra, 0x21C$ ((وهي تعني ضع العنوان الناتج في مسجل *return address*))

وبعدها نعود إلى العنوان المخزن داخل مسجل *ra* التي حفظناه سابقاً لنكمل العمل في السطر الذي يلي الاستدعاء.

ملاحظة: المبرمج يكتب التعليمات فقط ولا يحسب العناوين في حال اردنا القفز قفزات بعيدة.

أن عدد البتات المستخدمة لتخزين مسافة القفز في تعليمة jal تكون 20 بت و في تعليمة jalr تكون 12 بت فقط.

ولكن!!!!. إذا كان البرنامج ضخماً جداً ونريد القفز إلى دالة تبعد 4 ميجابايت مثلاً، فلن تتسع تعليمة jal العادية لتخزين هذا الرقم الكبير!

ويكون الحل باستخدام تعليمة auipc (add upper immediate to program counter) هذه التعليمة تأخذ مسافة مكونة من 20 بت، وتقوم بوضعها في الـ 20 بت العليا (Upper 20 bits)وتملأ الـ 12 بت المتبقية على اليمين بأصفار($12'b0$ ، ثم تجمع هذا الناتج مع عنوان عداد البرنامج الحالي pc وبذلك نكون قد قطعنا معظم المسافة الكبيرة.

عندما تكتب في الكود البرمجي أمر *call simple* وكانت الدالة بعيدة جداً (تتطلب عنواناً كاملاً بحجم 32 بت)، يقوم المترجم *(Assembler)* تلقائياً بتحويل هذه الكلمة الواحدة إلى تعليمتين متتاليتين داخل المعالج للوصول إلى الهدف على مرحلتين:

- *auipc ra, imm31:12*

تأخذ الجزء العلوي من العنوان (أول 20 بت من الـ 32 بت للعنوان الكامل) وتجمعه مع الـ *PC* الحالي، وتخزن النتيجة المؤقتة في مسجل العنوان المرجعي *ra*. الآن أصبحنا قريبين جداً من الهدف.

المرحلة الثانية تعديل المسافة والقفز النهائي ((تشييك عالمسافة))

- *jalr ra, ra, imm11:0*

تأخذ الجزء السفلي المتبقي من العنوان (الـ 12 بت الأخيرة)، وتجمعه مع العنوان المؤقت الموجود في *ra* لتصل بدقة متناهية إلى العنوان الفعلي للدالة، وتنفذ القفزة وتخزن في نفس الوقت عنوان العودة الصحيح في *ra*.

ملاحظة: دمجنا التعليمتين لحصلنا على 32 بت ((صرنا منقدر نفتل بالذاكرة براحتنا وين ماكنا 😆))

مثال توضيحي للفكرة السابقة:

إذا أردت السفر من سوريا إلى حي محدد في لبنان:

الخطوة الأولى: *(auipc)* تركب الطائرة من سوريا إلى مطار لبنان (قفزة هائلة تقربك من الهدف لكن لا توصلك لباب البيت).

الخطوة الثانية: *(jalr)* تأخذ تاكسي من المطار إلى الحي المستهدف بدقة (خطوة قصيرة ومحددة). هذا بالضبط ما تفعله الشريحة! لولا هذا التقسيم الذكي، لكان المعالج عاجزاً عن تشغيل البرامج والألعاب الضخمة التي تحتاج مساحات ذاكرة هائلة



بعض التعليمات الزائفة الأخرى في معمارية *RISC_V*:

Pseudoinstruction	RISC-V Instructions
j label	jal zero, label
jr ra	jalr zero, ra, 0
mv t5, s3	addi t5, s3, 0
not s7, t2	xori s7, t2, -1
nop	addi zero, zero, 0
li s8, 0x56789DEF	lui s8, 0x5678A addi s8, s8, 0xDEF
bgt s1, t3, L3	blt t3, s1, L3
bgez t2, L7	bge t2, zero, L7
call L1	auipc ra, $imm_{31:12}$ jalr ra, ra, $imm_{11:0}$
ret	jalr zero, ra, 0

See Appendix B for more pseudoinstructions.

مبدأ التصميم الرابع: التصميم الجيد يتطلب تنازلات (مقايضات) جيدة

تعدد صيغ التعليمات يسمح بالمرونة: فمثلاً تعليمات مثل *add*(جمع) و *sub*(طرح): تستخدم 3 معاملات *(Operand)* من نوع السجلات.*(Registers)* وتعليمات مثل *lw, sw*(تحميل وتخزين في الذاكرة) و *addi*(جمع مع ثابت): تستخدم معاملين من نوع السجلات ومعامل واحد ثابت *(Constant)*. ولكن عدد صيغ التعليمات يجب أن يظل صغيراً للالتزام بمبدأي التصميم الأول والثالث (البساطة تفضل النظام المنتظم، والحجم الأصغر يعني سرعة أعلى).

الخلاصة: الصيغ المتعددة تسمح للمعالج بتنفيذ وظائف متنوعة بكفاءة عالية حسب حاجة كل تعليمة.

Addressing modes

وتكون العنونة بالطرق التالية:

العنونة عبر المسجلات:

مثال في تعليمة $add\ s0, t2, t3$ جميع القيم موجودة داخل مسجلات

العنونة مع حقن عدد ثابت *immediate*:

مثال: في تعليمة $addi\ s4, t5, -73$ نلاحظ أن العدد غير موجود داخل مسجل وإنما تم وضعه بشكل مباشر في التعليمة

العنونة القاعدية *Base addressing*:

العنوان القاعدي: هو عنوان مرجعي ثابت أو متغير يتم تخزينه داخل أحد السجلات *(Registers)*. يعمل كـ "نقطة انطلاق."

تستخدم مع عمليات التحميل والتخزين.

حيث:

عنوان المُعامل *(Operand)* هو: العنوان القاعدي *(base address)* + القيمة الفورية *(immediate)*. (الإزاحة)

مثال: $sw\ t2, -25(t1)$

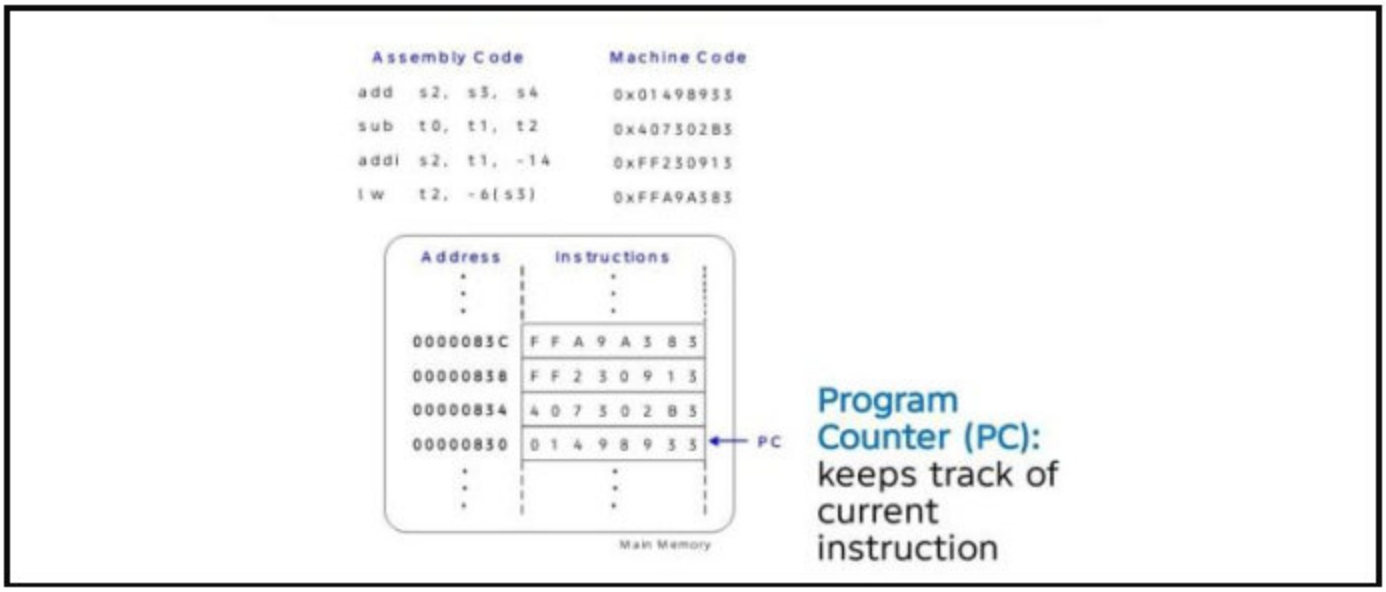
العنوان=قيمة المسجل *t1*-25

العنونة النسبية *(PC-Relative Addressing)*:

1. عند العنوان *0x354* (بالنظام الست عشري)، توجد تعليمة تحمل وسماً *(Label)* اسمه *L1*.
2. عند العنوان *0xEB0* (بعيداً في الأسفل)، توجد تعليمة تفرع مشروط *bne s8, s9, L1* وتعني: إذا كان *s8* لا يساوي *s9*، اقفز رجوعاً إلى المحطة *L1*.

$$Target\ Address = PC + Offset \quad \text{(نهائي - بدائي)}$$
$$0xEB0 - 0x354 = 0xB5C$$

مفهوم البرنامج المخزّن *(The Stored Program Concept)*:



في كل مرة نريد فيها تنفيذ مهمة جديدة، تقوم الحواسيب بتخزين التعليمات *(Instructions)* داخل الذاكرة الرئيسية *(Main Memory)* جنباً إلى جنب مع البيانات *(Data)*. المعالج يقرأ هذه التعليمات من الذاكرة، يفك شفرتها، ثم ينفذها.

وظيفة: الـ *PC*

هو سجل *(Register)* صغير داخل المعالج يحتوي دائماً على عنوان التعليمة الحالية التي يجب على المعالج تنفيذها الآن.

كيف يعمل؟

1. المعالج ينظر إلى الـ *PC* ليجد العنوان (مثلاً *00000830*).
2. يذهب للذاكرة ويجلب التعليمة الموجودة في هذا العنوان.
3. بعد تنفيذ التعليمة، يقوم المعالج تلقائياً بزيادة قيمة الـ *PC* ليؤشر إلى العنوان التالي (مثلاً *00000834*)، وهكذا يستمر البرنامج في العمل.

الترتيب الطرفي الصغير *Little endian*:

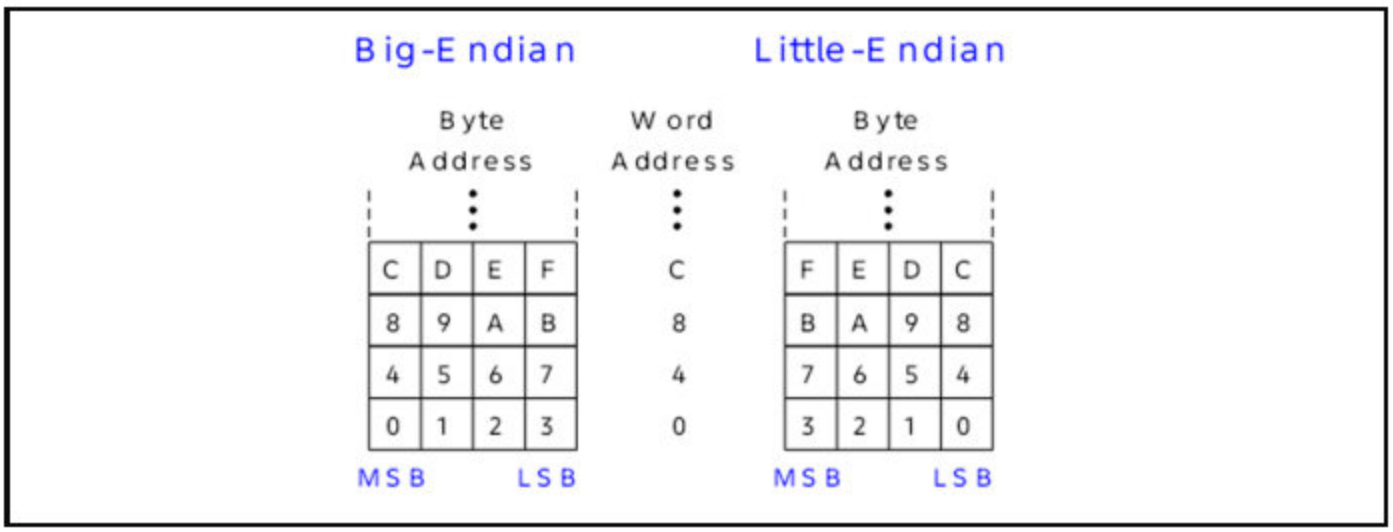
يخزن البايت الأصغر والأقل أهمية *(LSB)* في العنوان الأصغر للذاكرة أولاً.

الترتيب الطرفي الكبير *big endian*:

يخزن البايت الأكبر والأهم *(MSB)* في العنوان الأصغر للذاكرة أولاً.

ملاحظة: عنوان الكلمة (Word address) يكون متطابقاً وثابتاً في كلتا الحالتين سواء كان النظام Big-endian أو Little-endian.

ملاحظة 2: بمعالجات انتل نستعمل الترتيب الطرفي الصغير ومعالجات ريسك فايف غالبا ما تستعمل النمطين لذلك من أجل اختيار احد النمطين يوجد لدينا بت موجود في مسجل تحكم يمكن من خلاله اختيار أي نمط سنعمل



مثال من السلايدات:

- Suppose t0 initially contains 0x23456789
- After following code runs on **big-endian** system, what value is s0?
- In a **little-endian** system?

```
sw t0, 0(zero)
lb s0, 1(zero)
```

بفرض لدينا القيمة *0x23456789* ونريد وضعها في المسجل *t0*. ماهي قيمة المسجل *s0* في حالتين إن كان الترتيب الطرفي صغير أوكبير؟

الحل:

- Big-endian: s0 = 0x00000045
- Little-endian: s0 = 0x00000067

توضيح الحل:

نرى في الكود في السؤال أنه سيتم أولاً تخزين العنوان المعطى في المسجل *t0* ((الفكرة التمرين أنه نميز بين الترتيب الصغير وبين الترتيب الطرفي الكبير)) في حالة الترتيب الطرفي الكبير سيتم وضع الخانة الأكثر أهمية من يسار العنوان حتى يمينه ليكون بالشكل:

23 45 67 89

حيث 23 في الخانة الأكثر أهمية

نرى في التعليمة الثانية في الكود أننا نريد تحميل القيمة في المسجل *s0* مزاحاً بمقدار واحد ((مناخد يلي بعده مباشرة بالترتيب وبعد ال 23 عنا مباشرة 45 ومنه يتم المطلوب))

((ونفس الشرح مشان الترتيب الطرفي الصغير))

تعليمات الأعداد المؤشرة وغير المؤشرة

الضرب:

تعليمة الضرب المؤشر: *mulh*

تعليمة الضرب الغير مؤشر: *mulhu*: يعامل كلا المُعاملين *(Operands)* كأرقام بدون إشارة.

***mulhsu*: يعامل المُعامل الأول كقيمة بإشارة، والمُعامل الثاني كقيمة بدون إشارة.**

ملاحظة: عند ضرب عددين طولها 32 بت أطول ناتج سيكون طوله 64 بت وهذه مشكلة لأن جميع المسجلات في معمارية ريسك فايف طولها 32 بت من أجل حل المشكلة قمنا بتقسيم ناتج الضرب إلى قسمين:

سيتم إظهار 32 الدنيا عن طريق تعليمة mul

وسيتم إظهار ال 32 العليا عن طريق تعليمة mulhu (تكون الإشارات في هذا المسجل)

وعند ضرب عددين غير مؤشرين:

سيتم إظهار32 الدنيا عن طريق تعليمة mul

وسيتم إظهار ال 32 العليا عن طريق تعليمة mulhsu

مثال تطبيقي من السلايدات:

S1= 0x80000000; s2=0xc0000000;

Mulh s4,s1,s2	*Mulhu s4,s1,s2*	*Mulhsu s4,s1,s2*
Mul s3,s1,s2	*Mul s3,s1,s2*	*Mul s3,s1,s2*
$s1 = -2^{31}$; $s2 = -2^{30}$	$s1 = 2^{31}$; $s2 = 3x2^{30}$	$s1 = -2^{31}$; $s2 = 3x2^{30}$
$s1 \times s2 = 2^{61}$	$s1 \times s2 = 3x2^{61}$	$s1 \times s2 = -3x2^{61}$
s4 = 0x20000000	s4 = 0x60000000	s4 = 0xA0000000
s3 = 0x00000000	s3 = 0x00000000	s3 = 0x00000000

العمود الأول على اليسار:هو عملية ضرب لعددين مؤشرين

يكون ناتجهما 64 بت يتم تخزين 32 بت الدنيا في *s3* و 32 العليا في*s4* ونلاحظ أننا استخدمنا تعليمة *mulh*.

العمود الثاني: عملية ضرب غير مؤشر

لذلك استخدمنا تعليمة *mulhu* ونلاحظ أننا وضعنا ال 32 بت الدنيا في *mul* و 32 بت العليا في *mulhu*.

العمود الثالث: الضرب المختلط (مؤشر مع غير مؤشر)

عند استخدام تعليمة *mulhsu* يفهم المعالج أن المعامل الاول ((هون هو *s1*)) هو عدد مؤشر و*s2* عدد غير مؤشر ويتم وضع ال 32 بت الدنيا في *mul* و 32 بت العليا في *mulhsu*.

القسمة:

القسمة غير المؤشرة: نستخدم تعليمات مثل *divu,remu*

القسمة المؤشرة: نستخدم تعليمات مثل *div,rem*

التفرع *branch*:

للأعداد المؤشرة: *blt,bge*

للأعداد غير المؤشرة: *bltu,bgeu*

مثال:

Examples: s1 = 0x80000000; s2 = 0x40000000

blt s1, s2
s1 = -2^{31}; s2 = 2^{30}
taken

bltu s1, s2
s1 = 2^{31}; s2 = 2^{30}
not taken

نلاحظ أن الشرط محقق في التعليمة الأولى وبالتعليمة الثانية غير محقق

Set less than:

للأعداد المؤشرة: *slt,slti*

للأعداد غير المؤشرة: *sltu,sltiu*

مثال:

Examples: s1 = 0x80000000; s2 = 0x40000000

slt t0, s1, s2
s1 = -2^{31}; s2 = 2^{30}
t0 = 1

sltu t1, s1, s2
s1 = 2^{31}; s2 = 2^{30}
t1 = 0

slti t2, s1, -1 # -1 = 0xFFF
s1 = -2^{31}; imm = 0xFFFFFFFF = -1
t2 = 1

sltiu t3, s1, -1 # -1 = 0xFFF
s1 = 2^{31}; imm = 0xFFFFFFFF = 2^{32} - 1
t3 = 1

Load:

للأعداد المؤشرة:

تقوم بعمل تمديد للإشارة *(Sign-extend)* لإنشاء قيمة بحجم32 بت لتحميلها في السجل.

تحميل نصف كلمة *(Load halfwora)*: وتُكتب اختصاراً *lh*. وتحميل بايت *(Load byte)*: وتُكتب اختصاراً *lb*.

للأعداد غير المؤشرة:

تقوم بعمل تمديد بالأصفار *(Zero-extena)* لإنشاء قيمة بحجم 32 بت.

تحميل نصف كلمة غير مشار إليها *(Load halfword unsigned)*: وتُكتب اختصاراً *lhu*. وتحميل بايت غير مشار إليه *(Load byte unsigned)*: وتُكتب اختصاراً *lbu*.

كشف الطفحان العددي: Detecting Overflow

معمارية *RISC-V* لا توفر تعليمات جمع خاصة بالأرقام غير المؤشرة *(Unsigned)* ولا توفر تعليمات مخصصة لكشف الطفحان، لأن ذلك يمكن القيام به ببساطة باستخدام التعليمات الموجودة بالفعل.

ملاحظة: تم الأستغناء بشكل كامل عن الأعلام flags لتبسيط العتاد

كشف الطفحان غير المؤشر *(Unsigned Overflow)*:

الطفحان غير المؤشر يحدث عندما نجمع رقمين موجبين تماماً (بدون إشارة)، وتكون النتيجة أكبر من الحد الأقصى الذي يتسع له السجل *bltu t0, t1, overflow* تفرّع إلى *'overflow'* إذا كان الناتج *t0* أصغر من *t1* كـ *Unsigned*

مثال:

```
add  t0, t1, t2
bltu t0, t1, overflow
```

التعليمة *bltu* تعني *(Branch if Less Than Unsigned)* إذا كان الناتج *t0* أصغر من *t1*، فهذا دليل قاطع على حدوث طفحان، فيقفز المعالج مباشرة إلى مكان معالجة الخطأ *overflow*.

توضيح: اذا جمعت رقمين موجبين فمن الطبيعي أن يكون الناتج أكبر من أو يساوي أي منهما حيث إذا حدث طفحان، فإن القيمة "تدور" لتبدأ من الصفر مجدداً، وبالتالي ستصبح النتيجة أصغر من المدخلات

كشف الطفحان المؤشر *(Signed Overflow)*:

يحدث الطفحان في حالتين فقط:

1. جمع رقمين موجبين والنتيجة تخرج سالبة.
2. جمع رقمين سالبين والنتيجة تخرج موجبة.

```
Add t0, t1, t2

Slti t3, t2, 0      #t3=1 if t2 neg.

Slt t4, t0, t1      #t4=1 if result <t1

Bne t3, t4, overflow      #overflow if:

                          #t2 neg & result>=t1 or

                          #t2 pos & result<t1
```

الحالة الأولى إذا كان t2 موجباً:

السطر الثاني سيعطي .*t3 = 0* وبما أن *t2* موجب، فمن المفترض أن يكون الناتج *t0* أكبر من .*t1*

إذا حدث طفحان سيعطي السطر الثالث *t4 = 1* لأن الناتج سيصبح سالباً/أصغر

يحدث القفز *bne* ويُكتشف الطفحان أيضاً.

((توضيح من عندي 😜 : نحنا منعرف أنه اذا جمعنا عددين موجبين وطلعت النتيجة سالبة رح يصير في طفحان لما طلع بتاني سطر t3=0 يعني t2 قيمتها موجبة يعني لازم ناتج الجمع t0 يكون أكبر أو بساوي أحد العددين غير هيك صار عنا طفحان وهاد الشي يلي فحصناه بالكود بالسطر التالت ولما صارت t4=1 يعني أنه الناتج طلع أصغر من أحد الحدود وبأخر سطر فحص المسجلين انهم مختلفين وتم كشف الطفحان))

الحالة الثانية إذا كان t2 سالباً:

السطر الثاني سيعطي .*t3 = 1* وبما أن *t2* سالب، فمن المفترض أن يكون الناتج *t0* أصغر من .*t1* إذا حدث طفحان، سيعطي السطر الثالث *t4 = 0* لأن الناتج سيقلب موجباً/أكبر ((التوضيح هون نفس فوق بس بالعكس)). ويحدث القفز *bne* ويُكتشف الطفحان أيضاً.

ملاحظة: التعليمات المضغوطة compressed instruction:لضغط أي تعليمة Pنضع في بدايتها .cمثال: التعليمة العادية addيقابلها المضغوطة .c.add(يصبح طولها 16 بت بدلاً من 32).

Floating point registers

- *RVF* (32 بت)

8 بت للأس*(Exponent)* و 23 بت للجزء الكسري/الكسر*(Fraction)*

- *RVD* (64 بت)

11 بت للأس، و 52 بت للجزء الكسري.

- *RVQ* (128 بت)

15 بت للأس و112 بت للجزء الكسري

ملاحظة: عرض السجل (حجمه بالبت) يتحدد بناءً على أعلى دقة مدعومة في المعالج – على سبيل المثال، إذا تم تطبيق تمديد الدقة الرباعية(RVQ) ، فستكون السجلات بعرض 128 بت

الاستثناءات exceptions

تعريفها: تحدث عند تفعيل مقاطعة (استثناء بالهارد وير) أو ظهور خطأ (استثناء بالسوفت وير)

ماذا يحدث للمعالج في حالة الاستثناء؟

1. تسجيل سبب الاستثناء.
2. الانتقال (القفز) إلى دالة معالجة الاستثناء.
3. العودة إلى البرنامج الأصلي (بعد حل المشكلة).

مستويات الامتياز في RISC_V

- *Machine mode*نمط الآلة: التعامل المباشر مع العتاد دون وسيط وهو أقوى مستوى امتياز.
- *System mode (operating system)* أي نمط النظام: يصل إلى نظام التشغيل.
- *User mode (user program)*: نمط المستخدم :يصل إلى برامج المستخدم.
- *Hypervisor mode*: نمط المشرف الفائق (لدعم الأنظمة الوهمية / الافتراضية).

الفصل السادس: Microarchirecture

سندرس في هذا الفصل كيفية بناء المعمارية داخل العتاد الفيزيائي

من أجل هذا سنحتاج إلى تصميم جزئين:

Datapath:

يتمثل ب

- *ALU (Arithmetic Logic Unit)*: وحدة الحساب والمنطق التي تقوم بالجمع والطرح الفعلي.
- *Register File*: مجموعة السجلات التي تخزن البيانات مؤقتاً أثناء الحساب.
- *Program Counter (PC)*: السجل الذي يشير إلى عنوان التعليمة التالية.

Control signals:

تمثل عقل المعالج الذي يعمل على قراءة التعليمة الحالية *(Instruction)* وتفكيك شفرتها *(Decode)* ثم إصدار إشارات تحكم *(Control Signals)* عبارة عن نبضات كهربائية (0 أو 1).

هناك ثلاث طرق لتنفيذ المعمارية:

الدورة الأحادية *(Single-Cycle Microarchitecture)*

يتم تصميم مسار البيانات *(Datapath)* بحيث تدخل التعليمة وتخرج كاملة في نبضة ساعة واحدة أي *(1 Clock Cycle)* وهي بسيطة جداً في التصميم والفهم البرمجي والمنطقي ولكن يجب أن تكون دورة الساعة *(Clock Period)* طويلة بما يكفي لتستوعب أبطأ تعليمة في المعالج وهذا يعتبر أمر سلبي لأن التعليمات السريعة جداً ستنتهي سريعاً ثم يظل المعالج خاملاً ينتظر انتهاء الوقت الطويل للدورة مما يجعل التردد منخفض وبطيء

الدورات المتعددة *(Multicycle Microarchitecture)*

يعتمد على تقسيم زمن المعالج إلى دورات قصيرة جداً وسريعة. التعليمة الواحدة يتم تقسيمها إلى خطوات متتالية التعليمة السريعة تأخذ دورتين أو ثلاث، بينما التعليمة البطيئة تأخذ 5 دورات على سبيل المثال ومنه السريعة لم تعد مجبرة على انتظار وقت التعليمات البطيئة ولكن لمعالج لا يبدأ بالتعليمة الجديدة إلا بعد أن تنتهي التعليمة الحالية تماماً من كافة دوراتها، مما يعني أن أجزاء كثيرة من المعالج تظل غير مستغلة في أغلب الأوقات (ولكن يستهلك طاقة كبيرة).

ولحل هذه المشكلة يمكن استخدام التوجيه الأنبوبي.

التوجيه الأنبوبي(Pipelined Microarchitecture)

يتم تقسيم مسار البيانات إلى مراحل منفصلة يفصل بينها مسجلات مؤقتة .(Pipeline Registers) عندما تنتهي التعليمة الأولى من مرحلة "الجلب" وتنتقل لمرحلة "التفكيك"، تبدأ التعليمة الثانية فوراً في دخول مرحلة "الجلب".(الأسرع ولكن عتادها الأكثر تعقيداً)

زمن تنفيذ البرنامج:

زمن التنفيذ = (عدد التعليمات) × (عدد الدورات لكل تعليمة) × (عدد الثواني لكل دورة).

أي:

$$ExecutionTime = (\#instructions) \times (cycles/instruction) \times (seconds/cycle)$$

تعريفات:

- CPI (Cycles/instruction) : عدد دورات الساعة اللازمة لتنفيذ التعليمة الواحدة.
- clock period (seconds/cycle) : دورة الساعة أي الزمن المستغرق للدورة الواحدة بالثواني.
- IPC (instructions/cycle) :عدد التعليمات المنفذة في الدورة الواحدة وهو مقلوب الCPI .

التحدي يكمن في تلبية قيود ومحددات كل من:

1-Cost: التكلفة.

2-Power: الطاقة (استهلاك الطاقة).

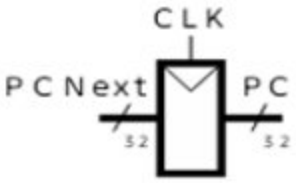
الحالة المعمارية

مفهوم" الحالة المعمارية (Architectural State) " يمثل المحتوى الكلي للذاكرة والسجلات الذي يحدد الوضع الحالي الدقيق للبرنامج أثناء تشغيله ." لو قمنا بإيقاف المعالج فجأة في جزء من المليون من الثانية، فإن المكونات الأربعة، التنفيذ يكون بطريقة الدورة الأحادية.

سنحتاج إلى أربع عناصر أساسية:

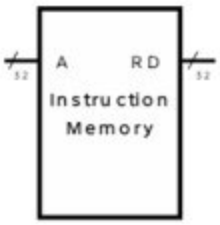
PC:

هو سجل *(Register)* فيزيائي صغير جداً، لكنه فائق الأهمية. وظيفته هي ارسال الإشارة إلى عنوان التعليمة الحالية في الذاكرة. مع كل نبضة ساعة *(CLK)*، يستقبل العنوان الجدي *(PC Next)*، والذي يكون عادةً عبارة عن *PC + 4* (لأن كل تعليمة حجمها 4 بايت/32 بت)، ليرسل العنوان الحالي للخطوة التالية.



ذاكرة التعليمات *(Instruction Memory)*

بمجرد أن يرسل الـ *PC* العنوان إلى المدخل *A*، تقوم ذاكرة التعليمات بالبحث السريع وإخراج شفرة التعليمة عبر المخرج *RD* هذه الشفرة هي التي ستحدد للمعالج ما يجب فعله (جمع، طرح، قفز، إلخ).

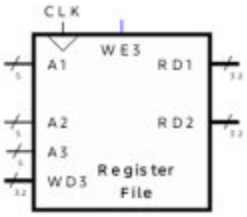


ملف السجلات

(Register File) بنية *RISC-V* تحتوي على 32 سجلاً عاماً للأعداد الصحيحة.

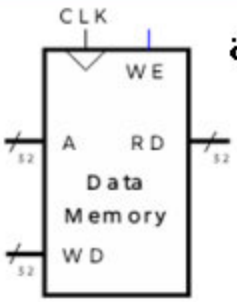
في حالة القراءة: التعليمة تحتاج لقراءة متغيرين؛ لذا يرسل المعالج أرقام السجلات المطلوبة عبر *A1* و *A2* حجمها 5 بت لأن $2^5 = 32$ فتخرج القيم فوراً من *RD1* و *RD2* لتذهب إلى وحدة الحساب والمنطق *(ALU)* لإجراء العملية الحسابية.

في حالة الكتابة: عند الرغبة في حفظ نتيجة حسابية في سجل معين، يتم إرسال رقم السجل عبر *A3*، والبيانات عبر *WD3*، وتفعيل إشارة السماح بالكتابة *WE3* مع نبضة الساعة.



ذاكرة البيانات *(Data Memory)*

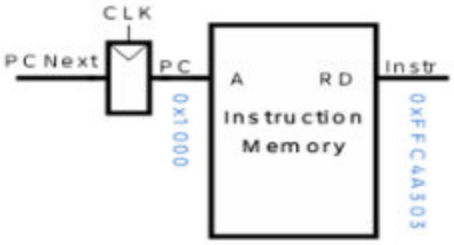
تُستخدم هذه الذاكرة عندما لا تكفي السجلات الـ 32 لتخزين بيانات البرنامج الكبيرة (مثل المصفوفات أو النصوص).





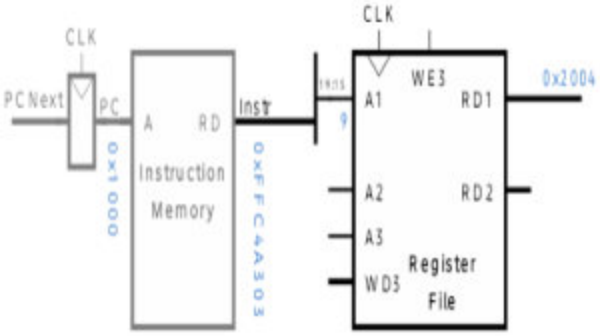
خطوات تصميم تعليمة Load Word بمعمارية RISC_V بطريقة الدورة الأحادية

الخطوة الأولى:



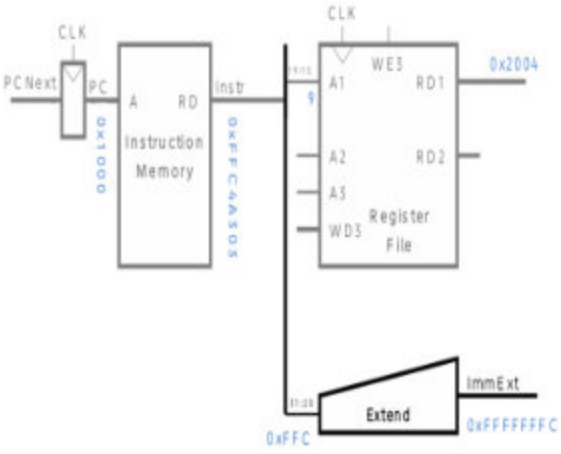
قراءة وفهم ماهو الأمر المراد تنفيذه. ويتم تحديد العنوان *(0x1000)*: يقوم عداد البرنامج *(PC)* بإرسال العنوان الحالي للتعليمة *(0x1000)* إلى مدخل الذاكرة المسمى *A* اختصار لـ *(Address)* جلب الأمر من الذاكرة: *(0xFFC4A303)* تبحث "ذاكرة التعليمات" عن الأمر المخزن في ذلك العنوان، وتخرجه عبر المخرج *RD* اختصار لـ *(Read Data)* وهو يمثل الشفرة البرمجية الخاصة بـ *lw* التي سيفهمها المعالج في الخطوات القادمة.

الخطوة الثانية:



بعد أن جلبنا أمر البرمجة في الخطوة الأولى، يحتاج المعالج الآن إلى تفكيكه وفهمه واستخراج البيانات المطلوبة لبدء العملية ويتم تحديد رقم السجل: *(9)* يأخذ المعالج جزءًا محددًا من شفرة الأمر (البتات من 15 إلى 19) لمعرفة مكان السجل المطلوب. قراءة محتوى السجل: *(0x2004)* يقوم "ملف السجلات" *(Register File)* بالبحث داخل السجل رقم 9، ويخرج القيمة المخزنة بداخله عبر المخرج *RD1* بيانات القراءة الأولى.

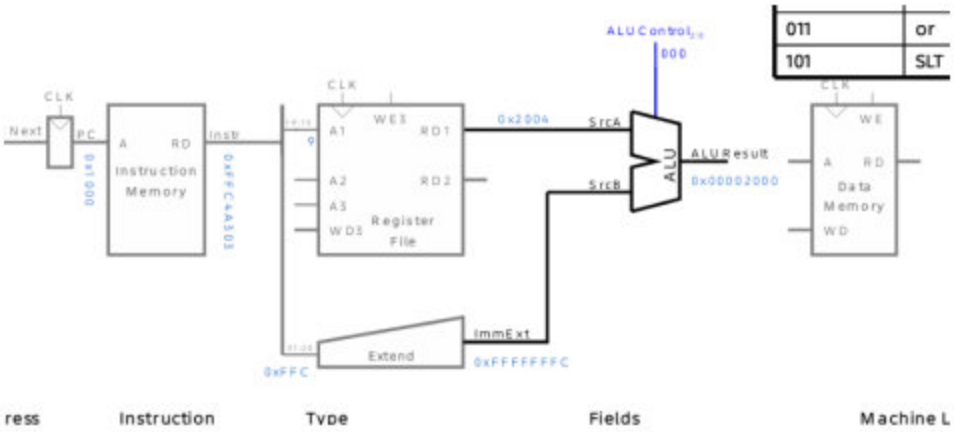
الخطوة الثالثة:



يأخذ المعالج جزءًا آخر من شفرة الأمر (البتات من 20 إلى 31) و بما أن مسار البيانات الداخلي للمعالج يعمل بحجم أكبر (مثلاً 32 بت)، لا يمكن للمعالج جمع رقم مكون من 12 بت مع رقم مكون من 32 بت مباشرة. هنا يأتي دور وحدة *Extend* التي تقوم بتوسيع الرقم الصغير بملء الخانات الفارغة جهة اليسار (مع الحفاظ على إشارته الموجبة أو السالبة). بعد مرور الرقم عبر وحدة التمديد، تحول إلى *0xFFFFFFFC* (رقم مكتمل بحجم 32 بت)، وهو جاهز الآن للدخول في الحسابات الرياضية في الخطوة التالية.

الخطوة الرابعة:

بعد أن جهز المعالج "العنوان الأساسي" و"قيمة الإزاحة" في الخطوات السابقة، جاء الدور الآن على وحدة الحساب والمنطق *(ALU)* لتدمجهما معاً ومعرفة العنوان الحقيقي الذي سنقرا منه البيانات من الذاكرة:



المدخل الأول: *(SrcA = 0x2004)* وهو العنوان الأساسي الذي قمنا بقراءته من ملف السجلات في الخطوة الثانية.

المدخل الثاني: *(SrcB = 0xFFFFFFFC)* وهو قيمة الإزاحة الممددة التي جهزناها في الخطوة الثالثة (والتي تكافئ رياضياً الرقم *4-* بالنظام العشري).

إشارة التحكم: *(000)* يرسل نظام التحكم إشارة قيمتها *000* تعني أمر الجمع *(add)*.

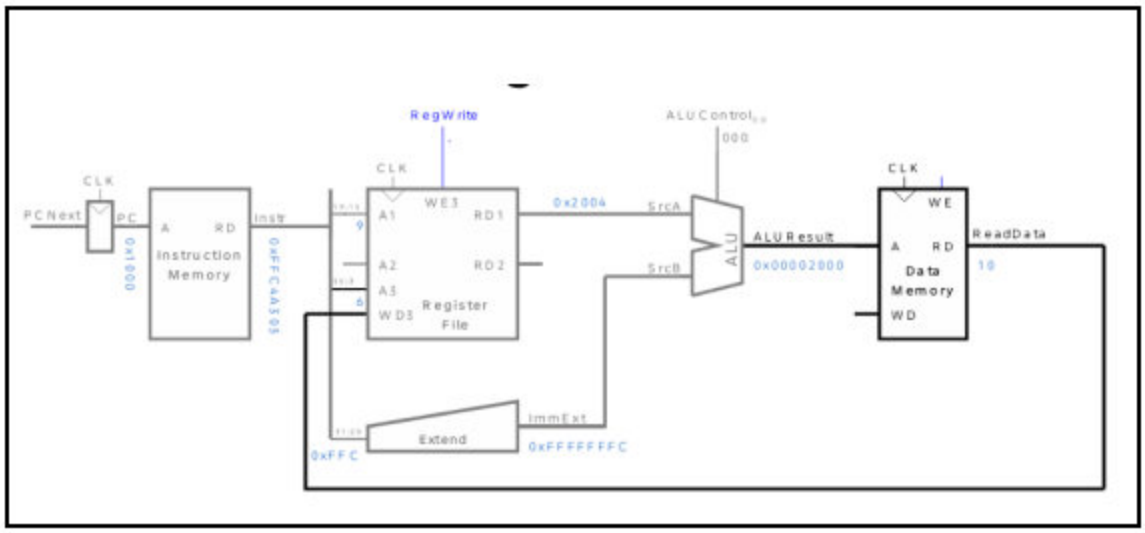
حساب الناتج: *(ALUResult = 0x00002000)* تقوم الـ *ALU* بجمع المدخلين معاً.

ملاحظة: يوضح الجدول إشارات التحكم لوحدة الحساب والمنطق:

ALUControl 2:0	Function
000	add
001	subtract
010	and
011	or
101	SLT

الخطوة الخامسة:

استقبلت "ذاكرة البيانات *(Data Memory)*" العنوان المحسوب من الخطوة السابقة وهو *0x00002000* عند المدخل *A*. قامت الذاكرة بالبحث عن محتوى هذا العنوان وأخرجت القيمة المخزنة داخله عبر المخرج *RD*. يظهر الناتج وهو الرقم *10* وفي نفس الوقت، يتم أخذ جزء من شفرة الأمر (البتات من 7 إلى 11) لتحديد السجل الذي سنحفظ فيه هذا الرقم. يظهر عند المدخل *A3* الرقم أي السجل رقم 6. يمتد مسار البيانات ليأخذ القيمة *10* المستخرجة من الذاكرة، ويعود بها إلى ملف السجلات ليغذي المدخل *WD3* بمجرد تفعيل إشارة التحكم *RegWrite* يتم رسمياً تخزين الرقم *10* داخل السجل رقم *6*، وبذلك تنتهي دورة تنفيذ الأمر *lw* بنجاح.



الخطوة السادسة:

حساب العنوان الجديد: *(PCPlus4)* تلاحظ في أسفل اليسار وجود وحدة جمع صغيرة *(Adder)* نشطة باللون الأسود العريض. تأخذ هذه الوحدة العنوان الحالي لـ *PC* وهو *(0x1000)* وتجمع عليه الرقم *4* ثابت القيمة.

الناتج الجديد: *(0x1004)* ناتج عملية الجمع هو *0x1004*، ويمثل عنوان السطر البرمجي التالي تماماً في ذاكرة التعليمات.

تغذية العداد: *(PC Next)* يتدفق هذا العنوان عبر المسار الأسود العريض ليدخل إلى منفذ *PC Next*. مع نبضة الساعة *(Clock Pulse)* القادمة، سيصبح هذا الرقم هو العنوان الحالي، وتبدأ الدورة من جديد من الخطوة الأولى لجلب أمر برمجي جديد.

